

回転機構のレーザー受光と楽器間無線 通信による同期・輪唱システム

情報工学科 2 年

24G1059

佐藤 涼菜

—— チーム 40 ——

宇井 祐真 (24G1021) 梅野 大誠 (24G1026)

山口 侑希 (24G1136) 薄井 悠希 (25G1020)

菊池 侑人 (25G1041)

2026 年 7 月 11 日

1 はじめに

1.1 社会的背景・課題・本稿の目的

近年、動画配信サイトやストリーミングサービスの普及によって、誰もが手軽に音楽コンテンツを視聴できる環境が整っている。特に日本では、ゲームやアニメなどのサブカルチャーを起点とした音楽コンテンツが数多く生み出されており、独自の文化として定着している。一方で、このようなデジタル配信の消費形態は、実際に会場へ足を運んで演奏を体感するライブイベントやコンサートとの相乗効果を生み、市場規模の拡大にも寄与している [1][2]。たとえば、博報堂の谷口による分析では、ストリーミングサービスの利用者がサービス内で新たに好みのアーティストを複数発掘し、そのアーティストたちを一度に鑑賞できるライブやフェスへ参加するという行動の流れが形成されつつあり、さらにライブでの体験を通じて別のアーティストと出会うことで、再びストリーミングでの視聴に戻るという音楽消費サイクルが生まれていると指摘されている [4]。すなわち、デジタル配信とライブイベントは互いの顧客を送客し合う関係にあり、この循環が音楽市場全体の拡大を後押ししていると考えられる。実際に、一般社団法人コンサートプロモーターズ協会が実施したライブ市場調査によれば、2025 年におけるライブ市場の年間売上高は 6443 億円、入場者数は 5999 万人にのぼり、デジタル配信の普及以降もライブ市場は堅調に拡大を続けている [3]。図 1 に示すように、国内のライブ・コンサート市場における年間売上額は 2010 年の 1280 億円から 2019 年の 3665 億円まで、10 年間でおよそ 2.9 倍に拡大している。2020 年には新型コロナウイルス感染症の影響により、会場に人を集めること自体が困難となった結果、年間売上額は 779 億円まで急落した。しかし、行動制限の緩和とともに市場は急速に回復し、2022 年には 3984 億円、2023 年には 5140 億円、そして 2025 年には過去最高となる 6443 億円を記録している。この推移から、ライブ会場に足を運ぶという体験に対する需要は、一時的な外的要因によって落ち込むことはあっても、長期的には一貫して拡大する傾向にあると言える。

デジタル配信が普及していく中で、なぜデジタル配信では代替できない価値がライブ会場に存在するのだろうか。その理由の一つとして、ライブ演出に用いられる音響や照明、ライブ参加者自身が感じる振動といった複数の物理的刺激が挙げられる。これらの刺激は、観客一人ひとりに個別に届けられるのではなく、同じ空間にいる観客全員に同時に共有されるという性質を持つ。その結果、観客同士が同じ体験を共有しているという感覚、すなわち空間全体としての一体感が形成される。換言すれば、ライブ会場における没入感とは、ユーザが音楽を聴くだけでとどまらない、物理的、視覚的な技術と同じ空間で同時に受け取ることによって生まれる体験だと考えられる。この仮説は、複数の研究によっても裏付けられている。ライブ演奏とデジタル配信を直接比較した実証研究によっても裏付けられている。Kreuzer ら (2025) は、同一のクラシック音楽コンサートを、会場で聴取する条件と映像配信で視聴する条件とで比較する実験を行った。その結果、没入感や社会的つながりの感覚を含む複数の評価軸において、ライブ条件の方が配信条件よりも有意に高い評価を得ることを示した [5]。すなわち、同じ演奏内容・同じ音質であっても、同じ空間を他の観客

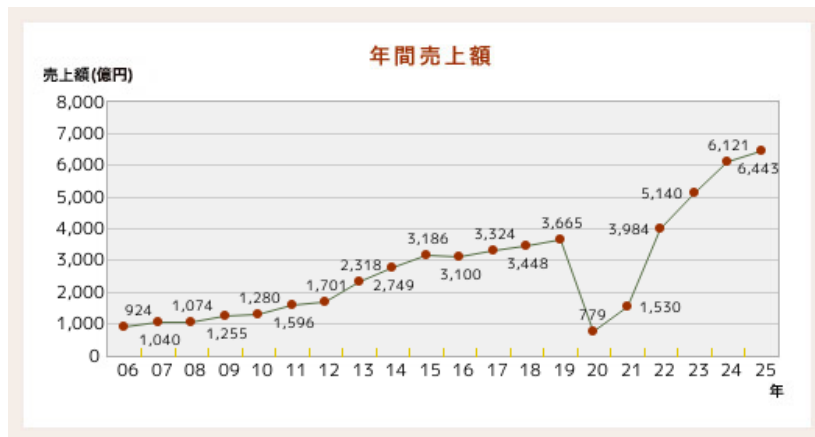


図1 国内ライブ・コンサート市場における年間売上額の推移（一般社団法人コンサートプロモーターズ協会「基礎調査推移表」より）

と共有しているという感覚そのものが、没入感を左右する独立した要因であり、これは録画・配信技術の高度化だけでは補えないことを意味する。したがって、音そのものの再現性だけでなく、ライブ体験に固有の空間的フィードバックを組み込むことで没入感の高い演奏体験の実現に有効であると考えられる。

また、Natalia Cooper ら（2018）は、仮想現実（VR）環境における体験の再現度を高めるため、視覚情報のみでは表現が難しい感覚を、音や触覚など別の感覚を用いた代替フィードバックとして提示する実験を行った。その結果、視覚刺激のみを提示した場合と比較して、音や触覚といった複数の異なる刺激を組み合わせると提示した場合の方が、利用者の没入感・関与感が有意に高まることを明らかにした [6]。つまり、複数の感覚情報を同時に提示する多感覚的フィードバックは、ユーザの体験における没入感の向上に有効であると考えられる。また、Human-Computer Interaction（HCI）分野の知見も同様の傾向を支持している。Martin ら（2024）は、ライブコンサート環境を対象とした研究において、観客が演奏を受動的に聴取するだけでなく、能動的に体験へ参加することが、会場全体の活気や観客の感情の高まりに寄与すると結論づけた [7]。よって、ユーザ自身が能動的に関与できる仕組みもまた、システムの没入感を高める要因として重要であると考えられる。

一方、音楽を自動で演奏する自動演奏技術の分野では、デジタル技術の発展により高精度な同期や楽曲再現が可能になっている。たとえば、Yamaha が提供する Disklavier ENSPIRE シリーズは、光学センサとサーボモータを用いて演奏情報を高精度に再現するとともに、鍵盤が実際に動作する様子をユーザへ視覚的にフィードバックを行う機能を備えている [8]。しかし、この視覚的フィードバックは、ピアノの鍵盤という限られた範囲でのみ提示されるものであり、その場に居合わせたユーザ1人のみが確認できる情報にとどまる。したがって、複数人が同じ空間にいる状況を想定すると、自動演奏技術によって提示される視覚的フィードバックは、空間全体には行き渡らないという限界を抱えていると言える。

以上を踏まえると、現状の自動演奏技術は、演奏そのものの精度は高い一方で、前述した「多感覚的フィードバック」と「空間全体への共有」という、没入感を高めるうえで重要な要素を十分に

は満たせていないという課題が生じる。特に、Kreuzer らの知見が示すとおり、同じ場を共有しているという空間的なフィードバックは、音質や演奏精度をいくら高めても代替できない、ライブ体験に固有の要素である。したがって、デジタル配信や高精度な自動演奏技術がどれほど発展しても、この空間的フィードバックを欠いたままでは、ライブ会場に匹敵する高い没入感を得ることはできないと結論づけられる。そこで本稿では、自動演奏技術に、ライブ体験に共通するこの空間的フィードバックを付加した楽器間通信システム「回転機構のレーザー受光と楽器間無線通信による同期・輪唱システム」を提案する。本システムは、ユーザの動作を検知する指揮デバイス、指揮デバイスからの情報に応じ回転する観覧車型の中継機デバイス、そして中継機からのレーザー光を受光し演奏を行う楽器デバイスによって構成される。指揮デバイスによるユーザの能動的な動作、観覧車型デバイスによる空間全体への視覚的なフィードバックの共有、そして高精度な音色の演奏による聴覚的な刺激を組み合わせることで、従来の自動演奏技術では実現が難しかった、会場全体を巻き込む没入感の高い演奏体験の提供を目指す。

1.2 類似技術・先行研究

本節では、本システムの独自性を明確にするため、音楽に視覚的なフィードバックを付加する既存技術を整理し、それぞれの特徴と限界を検討する。

第一に、DMX512 信号（以下、DMX 信号と表記する）を用いた照明制御システムが挙げられる。DMX 信号とは、舞台照明の操作卓と調光機との間でデータをやり取りするための通信規格であり、現在では舞台演出やコンサートにおいて、音楽に合わせた光量制御を行う目的で広く利用されている [9]。このシステムは、音楽という聴覚的な刺激に加えて光という視覚的な刺激を提示することで、観客の没入感を高めることができる。しかし、DMX 信号による照明制御は、あらかじめプログラムされた演出を再生する仕組みであるため、観客はその光の変化を一方的に受け取る、すなわち受動的に認識する立場にとどまる。前節で述べたとおり、能動的な関与が没入感を高める要因の一つであることを踏まえると、観客が演奏そのものに関与できない DMX 信号による演出には限界があると言える。この課題に対し、本提案では指揮デバイスを導入し、利用者が指揮という身体動作を通じて演奏そのものを能動的に制御できるようにすることで、身体的な関与に基づく没入感の向上を狙う。

第二に、物理的な操作によって音楽を制御するタンジブル・インターフェースの例として、ReacTable が挙げられる。ReacTable は、卓型のディスプレイ上に複数の物理モジュールを配置し、その配置や動きに応じて音を合成するシンセサイザの一種であり、利用者は身体的な物理操作を通じて演奏を制御するとともに、ディスプレイに表示される視覚的な情報によって演奏状態のフィードバックを受け取ることができる [10]。しかし、この視覚的フィードバックはディスプレイという限られた表示領域内で完結しており、操作を行う利用者本人以外の第三者や、会場全体へフィードバックを共有する仕組みとしては設計されていない。そこで本提案では、指揮デバイスに加えて観覧車型の中継機デバイスを導入する。観覧車型デバイスが演奏情報に応じて回転する様子を、利用者本人だけでなく周囲にいる第三者も視認できるようにすることで、演奏状態の把握や共有を空間全体に広げ、多感覚的なフィードバックを実現することを狙う。

以上のように、DMX 信号による照明制御は観客の能動的な関与を欠き、ReacTable のような既存のタンジブル・インターフェースは視覚的フィードバックが個人の操作範囲にとどまるという、それぞれ異なる限界を抱えている。本提案は、指揮デバイスによる能動的な演奏制御という既存研究の知見と、観覧車型デバイスによる空間共有性という要素を組み合わせることで、これら二つの課題を同時に解決し、空間全体への多感覚的なフィードバックを通じて、利用者の没入感および関与感を高めることを目指す。

2 作成したシステムの概要

本節では、作成したシステムの概要について説明する。本システムは、ユーザの能動的な動作と連動した観覧車型中継機デバイスの回転およびレーザーの発光を通じて、演奏空間全体に多感覚なフィードバックを与えることで、従来の自動演奏技術における演奏インターフェースでは得られない高い没入感をユーザに提供することを目的として設計されている。まず、本システムの全体の構成図を図2に示す。本システムは、指揮デバイス、観覧車型の中継機デバイス、楽器デバイスという3つのデバイスから構成される。ユーザが指揮デバイスに取り付けられたボタンを押下すると、同期開始情報が無線通信 (UDP) によって中継機デバイスおよび楽器デバイスへ送信され、中継機デバイスの回転が開始する。中継機デバイスは常にレーザー光を照射しており、回転するレーザー光を楽器デバイスの CdS セルが受光することで楽器間で同期通信が開始される。楽器間の同期通信では、まず各楽器デバイスがレーザー光の受光間隔から BPM を推定し、その推定値が安定するまで待機する。推定が安定すると、ドラムを担当する楽器デバイスをマスター、ピアノ・ストリングス・フルートを担当する楽器デバイスをスレーブとした UDP 通信により、スレーブがマスターへ参加を通知したうえで両者の発音タイミングのずれを相互に送受信し合い、発音タイミングを補正する。全ての楽器デバイス間でこのずれが閾値内に収まり同期が確立すると、各楽器デバイスは自身が保持する楽譜データに基づき、発音タイミングごとに USB シリアル通信で音程や音量等の発音情報を PC へ送信する。PC 上で動作する Processing は、受信した発音情報に従って実際の演奏音を生成・出力することで、演奏が開始される。なお、この同期通信および発音処理の詳細については 2.3.2 項（楽器デバイスの内部設計）で説明する。また、演奏開始後にて楽曲の BPM を変更したい場合、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、中継機デバイスへ UDP 通信がなされ回転速度が変化する。そして、この回転速度に伴い変化した受光間隔によって各楽器デバイスが USB シリアル通信を行うことで、接続された PC から演奏されている楽曲の BPM が変更される。最後に、演奏開始後にて楽曲の音量を変更したい場合、ユーザがスライド式可変抵抗器を操作してうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、直接楽器デバイスへ UDP 通信を行い楽曲の音量を変更する。ここからは各デバイスの概要について説明する。

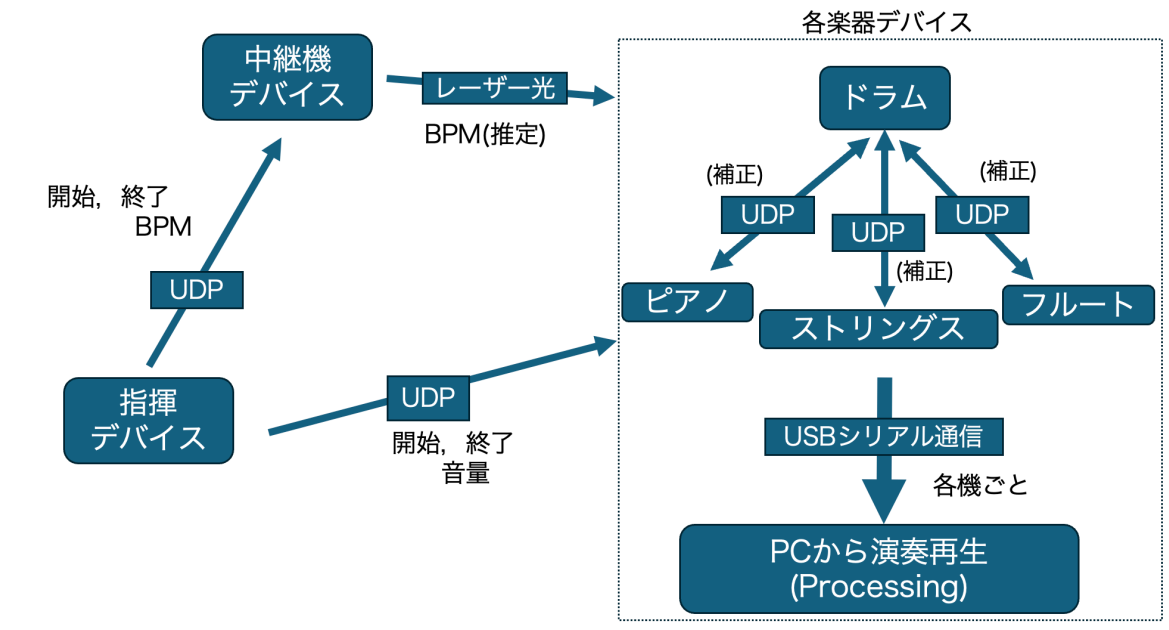


図 2 全体の構成図

2.1 指揮デバイス

2.1.1 概要

まず、指揮デバイスについて説明する。指揮デバイスのシステム構成図を図 3 に示す。本デバイスは、演奏全体を統括するデバイスであり、主に以下の 3 つの機能を有する。

第一の機能は、演奏開始トリガーの送信である。本機能の設計について図 4 に示す。ユーザが指揮デバイスに搭載されたタクトスイッチを押下すると、中継機デバイスと楽器デバイスへ演奏開始トリガーを送信する。さらに、中継機デバイスの回転が開始される。

第二の機能は、楽曲の BPM 変更である。本機能の設計について図 5 に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると新しい設定 BPM 情報が中継機デバイスへ送信される。これにより、中継機デバイスのモータ回転速度がこの値に追従して変化する。そして、その回転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、BPM の変更される。また、楽曲の BPM は 5 段階に分けて変更できる。

第三の機能は、楽曲の音量変更である。本機能の設計について図 6 に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると音量情報が各楽器デバイスへマルチキャスト送信されることで、音量が変更される。また、BPM と同様に音量も 5 段階に分けて変更できる。ここからは、外部設計と内部設計について説明をする。

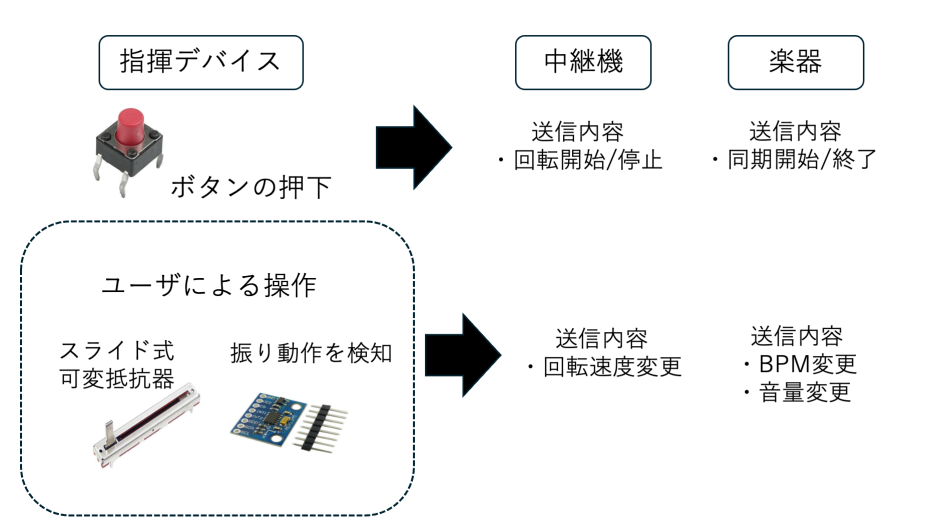


図3 指揮デバイスのシステム構成図

2.1.2 外部設計

本節では指揮デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表1に示す。デジタル3軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器はBPMおよび音量変更に使用される。当初はデバイスの電力供給源として9Vのアルカリ電池を外部電源として使用することを検討していたが、電池なしでも正常動作が確認できたため、軽量化のためPCとのUSB接続から電力を供給するように変更した。Arduino内蔵の電圧レギュレータにより5Vおよび3.3Vを生成し、各モジュールへ安定した電力を供給する。また、指揮デバイスの回路図を図7に示す。加速度センサにはGY-291(ADXL345)を用いており、I2C通信によりArduinoと接続する。CSピンをArduinoの3.3Vに接続することでI2Cモードに設定し、さらにSDOピンをGNDに接続することでI2Cアドレスを0x53に固定している。また、SDAピンおよびSCLピンはそれぞれデータ通信線およびクロック信号線としてArduinoのA4、A5ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1ピンからArduinoのD2ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用ArduinoとのUDP通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため10kΩのプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピンD3はプルアップ抵抗を介して5Vに接続されるためHIGH状態となる。一方、ボタン押下時には回路がGNDに短絡され、入力電位がLOWに遷移する。指揮デバイスのプログラムではD3ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

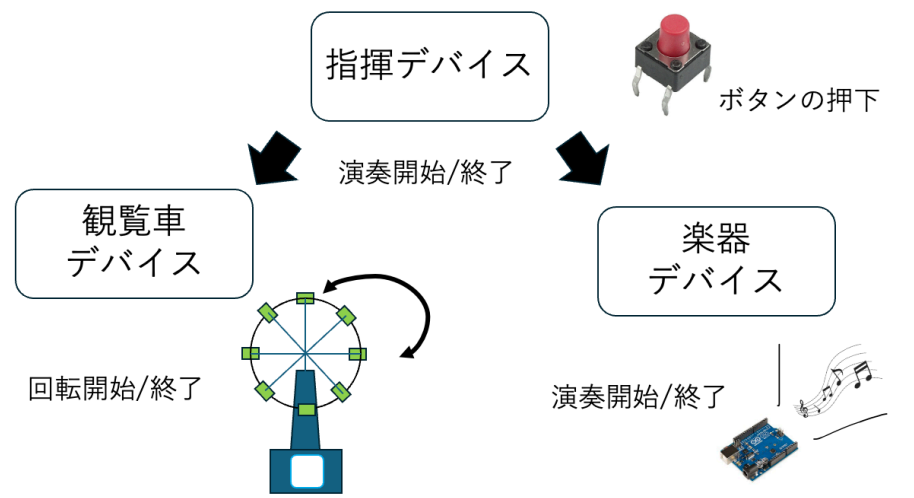


図 4 演奏開始トリガー送信機能設計

表 1 指揮デバイスの構成部品

機材	品番	個数 [個]
Arduino UNO R4 WiFi	-	1
スライド式可変抵抗器 10 k Ω	-	2
デジタル 3 軸加速度センサ	GY-291	1
タクトスイッチ	DTS-63	1
抵抗器 10k Ω	-	1
ジャンパーワイヤー	-	適宜

図～～に実際に作成した指揮デバイスを示す。実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用した。また、内部にはブレッドボードに加速度センサ、スライド式可変抵抗器、タクトスイッチを設置する。デバイスのユーザビリティを向上させるため、ユーザが操作するスライダー部分やボタンは外部へ露出させる構造を採用し、さらにデバイスの先端部に加速度センサを設置することで操作時の重心を安定させる。

2.1.3 内部設計

本節では、指揮デバイスの内部設計について説明する。まず、指揮デバイス全体の内部動作を明確にするため、シーケンス図を図 8 に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知し Arduino に送信される。この時、値が閾値を超過した場合、観覧車デバイスと無線通信を開始し観覧車デバイスが送信した演奏情報を基に回転する。また、ボタンを押下した時、各楽器デバイスへ楽曲の初期 BPM および音量情報が送信されることで演奏が開始される。

次に、指揮デバイスの制御プログラムのファイル構成を示す。表 2 の通り、各ファイルは機能ごとにクラス化されている。

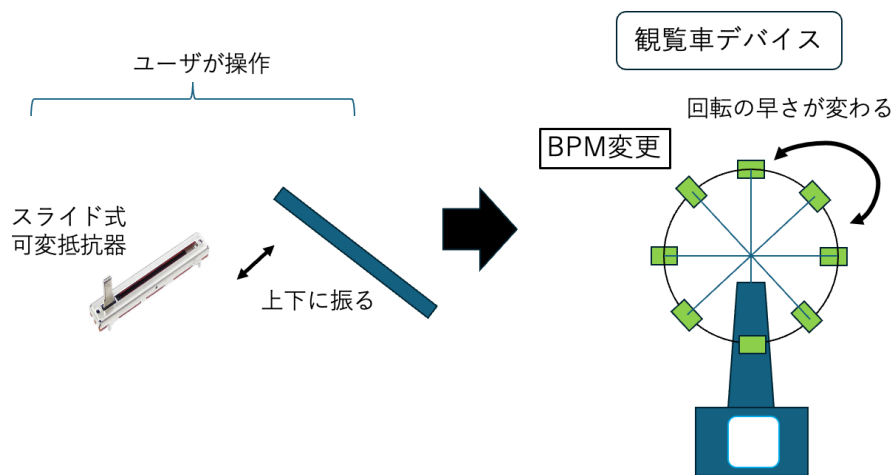


図 5 BPM 変更機能設計

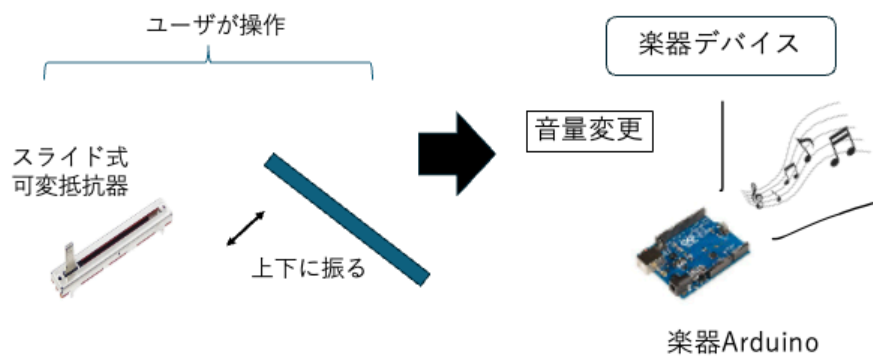


図 6 音量変更機能設計

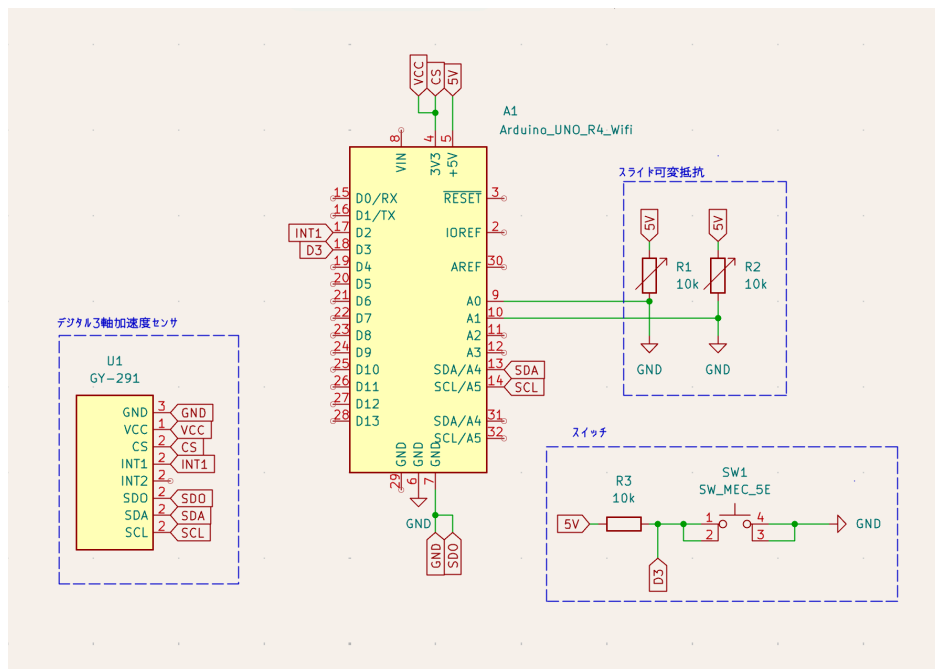


図7 指揮デバイスの回路図

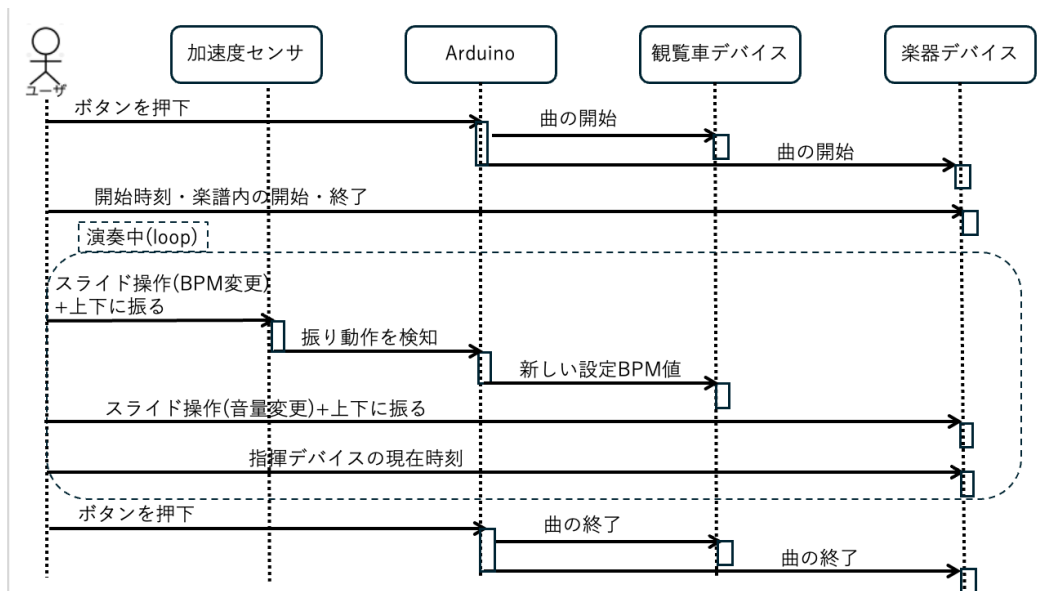


図8 指揮デバイスのシーケンス図

表 2 指揮デバイスのファイル構成

ファイル名	概要
Controller_simultaneous.ino	setup/loop による全体制御, ボタン入力の判定
AccManager.h / AccManager.cpp	加速度センサ (ADXL345) の初期化, 振り動作の検知, I2C バス復旧処理
PotManager.h / PotManager.cpp	スライド式可変抵抗器 2 系統の移動平均処理および BPM・音量段階の判定
SyncManager.h / SyncManager.cpp	Wi-Fi/UDP 通信の初期化, 各デバイスへの冗長送信処理

Controller_simultaneous.ino の loop 関数は, ボタン判定を行う button 関数, 可変抵抗器を処理する PotManager, 加速度センサを処理する AccManager を毎ループ順に呼び出す構成となっている. loop 関数をコード 1 に示す. プログラム全文は付録に示す.

コード 1 Controller_simultaneous.ino の loop 関数

```

1 void loop() {
2     button();
3     pot.update();
4     acc.update(sync, pot);
5 }

```

ここからは, ボタン入力の判定, 加速度センサの制御, 可変抵抗器を用いた BPM・音量読み取り機能の順に, 使用する変数・関数の仕様と処理内容について説明する.

まず, 指揮デバイスにおけるボタン入力の判定処理について説明する. ボタンの誤判定を防ぐため, プルアップ抵抗を用いた回路を構成しており, ボタンが押されていない状態ではデジタルピンに 5V が印加され, 押下時には回路が GND に短絡されることでピンの電位が LOW へ遷移する. 指揮デバイスのプログラムでは, このデジタルピンの電圧状態の変化を常時監視することでボタンの押下を判定する. さらに, チャタリングによる誤判定や冗長パケットによる誤作動を防止するため, 押下検知後に一定時間の待機処理を設けるとともに, 演奏状態管理フラグ (isButtonPlaying) を用いたトグル制御を導入している. これにより, 同一の物理ボタンのみで演奏の開始と終了を交互に切り替えることが可能となる. ボタン入力の判定で使用する変数の仕様を表 3 に示す.

表 3 ボタン入力の判定で使用する変数の仕様一覧

変数名	型	説明
input_PIN	const uint8_t	ボタンの状態を読み取るデジタルピン番号を定義
switch_status	bool	現在読み取ったボタンピンの状態を格納
before_switch_status	bool	前回読み取ったボタンピンの状態を格納 (立下り検知に使用)
isButtonPlaying	bool	演奏状態管理フラグ. トグル制御により演奏中 (true) / 停止中 (false) を保持

続いて、ボタン入力の判定で使用する関数の仕様を表 4 に示す。

表 4 ボタン入力の判定で使用する関数の仕様一覧

関数名	戻り値	引数	説明
button	void	なし	input_PIN の立下り (switch_status が 0 かつ before_switch_status が 1) を検知すると isButtonPlaying をトグルし、開始時は現在の BPM 段階番号を付与して SyncManager の sendStart 関数を、終了時は sendEnd 関数を呼び出すことで、各楽器機・観覧車デバイスへ演奏開始・終了信号を送信する。送信後は 300 ms の待機によりチャタリングを防止する。

button 関数はこの 1 関数のみで構成されるため、以下では処理の流れに沿って実際のコードを 2 箇所に分けて説明する。まず、立下りエッジの検知部分について説明する。switch_status に現在のピン状態を読み込み、前回値 before_switch_status と比較することで、HIGH → LOW へ変化した瞬間 (ボタンが押された瞬間) のみを検知する。該当部分をコード 2 に示す。

コード 2 button 関数 (立下りエッジの検知部分)

```
1 switch_status = digitalRead(input_PIN);
2 if (switch_status == 0 && before_switch_status == 1) {
3     ...
4 }
5 before_switch_status = switch_status;
```

次に、立下りを検知した際の状態トグルと送信処理について説明する。isButtonPlaying が true (演奏中) であれば sync.sendEnd() を呼び出して演奏を終了し、false (停止中) であれば現在の BPM 段階 (pot.getBpmStep()) を引数として sync.sendStart() を呼び出して演奏を開始する。いずれの場合も isButtonPlaying の値を反転させたうえで、delay(300) により 300 ms 待機することでチャタリングを防止する。該当部分をコード 3 に示す。

コード 3 button 関数 (状態トグルと送信処理部分)

```
1 if (isButtonPlaying) {
2     sync.sendEnd();
3     isButtonPlaying = false;
4 } else {
5     uint8_t step = pot.getBpmStep();
6     sync.sendStart(step);
7     isButtonPlaying = true;
8 }
9 delay(300); // チャタリング防止
```

なお、sendStart・sendEnd の内部では、SyncManager クラスが複数台のデバイスへラウンドロビン方式による冗長送信を行っている。この通信処理の詳細な実装は付録に示すプログラム全文を参照されたい。

次に、指揮デバイスにおける加速度センサの制御処理について説明する。指揮デバイスは、加速度センサを用いてユーザが指揮棒を振ったことを検知し、演奏情報の更新を行う。センサ値の急激な変化を読み取ることで動作を判定し、誤検知防止機能を設けることで安定した動作を実現している。処理の流れは、差分算出と判定、多重検知の防止、状態の反転、事後処理の4段階に大別される。まず差分算出と判定では、現在の合成加速度 currentAccVal と前回値 lastAccVal との差の絶対値を求める。この差分が検知閾値 accThreshold を超過し、かつ前回検知から shakeInterval 以上の時間が経過している場合に、指揮棒を振ったと判定する。多重検知の防止では、一度振る動作を検知すると lastDetectTime を現在時刻に更新し、以降 shakeInterval が経過するまでは加速度の変化を無視することで、一連の動作による複数回の誤検知を防止する。状態の反転では、振る動作を検知するたびに isStarted の bool 値を反転させる。事後処理として、次回の判定に備えて currentAccVal の値を lastAccVal へ代入し、前回のサンプリング時刻を更新する。また、振る動作の確定時には、可変抵抗器側で設定値の変更を検知したフラグ bpmFrag または volFrag (2.1.3 項で説明する) が有効になっている場合、SyncManager クラスを介してその情報の送信指示を行う。さらに、I2C 通信の異常を検知した場合には自動復旧を行う機能を備えている。加速度センサの制御で使用する変数の仕様を表5に示す。

表 5 加速度センサの制御で使用する変数の仕様一覧

変数名	型	説明
currentAccVal	float	加速度センサから算出した合成加速度の値を格納
lastAccVal	float	変化量算出のための直前の加速度センサ値を格納
accThreshold	const float	振ったと判断するための変化量の閾値を定義
lastDetectTime	unsigned long	前回振ったことを検知した時間を ms 単位で格納
shakeInterval	const unsigned long	連続検知を防止するための待機時間を定義
lastSampleTime	unsigned long	前回センサ値をサンプリングした時間を ms 単位で格納
ACC_SAMPLE_INTERVAL	const unsigned long	加速度センサ値をサンプリングする周期を定義
isStarted	bool	楽器の演奏状態を保持 (true: 演奏中/false: 停止中)
ADXL345_ADDR	const uint8_t	加速度センサ (ADXL345) の I2C アドレスを定義
REG_POWER_CTL	const uint8_t	加速度センサの電源管理レジスタのアドレスを定義
REG_DATA_FORMAT	const uint8_t	測定範囲設定用レジスタのアドレスを定義
REG_DATAX0	const uint8_t	測定値が格納される先頭レジスタのアドレスを定義
lastRecoverMs	unsigned long	I2C 復旧処理の連続実行を防ぐための最終復旧時刻を保持

続いて、加速度センサ制御 (AccManager クラス) で使用する関数の仕様を表 6 に示す。

表 6 加速度センサ制御 (AccManager クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
setupADXL345	void	なし	ADXL345 の電源投入と測定レンジの設定を行い、初期の加速度値を lastAccVal に格納する初期化関数
readAccMagnitude	float	なし	ADXL345 から X/Y/Z 軸の生の値を取得し、2 乗和の平方根を返す。I2C 通信異常を検知した場合は直前の加速度値を返す
recoverI2C	void	なし	I2C バスのロックアップ異常を検出した場合に、SCL 線のクロッキングによるバスクリアと STOP 条件の生成、Wire およびセンサの再初期化によって通信を復旧する
update	void	SyncManager& sync, PotManager& pot	ACC_SAMPLE_INTERVAL ごとに加速度値を取得し、差分算出・多重検知防止判定・状態の反転・事後処理までの一連の振る動作検知処理を行う。loop 内で毎回呼び出す
updateShakedInfo	void	SyncManager& sync, PotManager& pot	振る動作の確定時に呼び出され、PotManager の bpmFrag / volFrag が立っている場合に SyncManager 経由で BPM・音量の段階番号を送信する

以下では、表 6 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、setupADXL345 関数について説明する。この関数は、Wire.begin による I2C 通信の開始後、ADXL345 の POWER_CTL レジスタに Measure ビットを書き込むことでセンサを測定モードへ移行させ、DATA_FORMAT レジスタを初期値のまま設定する。setupADXL345 関数をコード 4 に示す。

コード 4 AccManager::setupADXL345 関数

```

1 void AccManager::setupADXL345() {
2     Wire.begin();
3
4     Wire.beginTransmission(ADXL345_ADDR);
5     Wire.write(REG_POWER_CTL);

```

```

6 Wire.write(0x08); // Measure bit ON
7 Wire.endTransmission();
8
9 Wire.beginTransaction(ADXL345_ADDR);
10 Wire.write(REG_DATA_FORMAT);
11 Wire.write(0x00);
12 Wire.endTransmission();
13
14 lastAccVal = readAccMagnitude();
15 }

```

次に、加速度の実測値を取得する readAccMagnitude 関数について説明する。この関数は、DATA0 レジスタから連続する 6 バイト (X, Y, Z 軸それぞれ 2 バイトの符号付き整数) を requestFrom で読み出し、3 軸のベクトル合成値 (合成加速度) を平方根の計算により求めて返り値とする。readAccMagnitude 関数をコード 5 に示す。

コード 5 AccManager::readAccMagnitude 関数

```

1 float AccManager::readAccMagnitude() {
2   Wire.beginTransaction(ADXL345_ADDR);
3   Wire.write(REG_DATA0);
4   if (Wire.endTransmission(false) != 0) {
5     recoverI2C();
6     return lastAccVal;
7   }
8
9   uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
10  if (n < 6) {
11    recoverI2C();
12    return lastAccVal;
13  }
14
15  int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8));
16  int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8));
17  int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8));
18
19  return sqrtf((float)rawX*rawX + (float)rawY*rawY + (float)rawZ*rawZ);
20 }

```

ここで、Wire.endTransmission(false) がエラーを返した場合、または requestFrom で 6 バイト読み出せなかった場合には、I2C バスがロックアップ (スレーブが SDA 線を握ったまま停止する現象) していると判断し、recoverI2C 関数を呼び出したうえで前回値 lastAccVal を返すことで、誤ったシェイク検知 (ニセ shake) を防いでいる。続いて、recoverI2C 関数について説明する。この関数は、SCL 線を 9 回手動でトグルさせて SDA 線の握りを解放したうえで STOP 条件を生成し、Wire を再初期化して ADXL345 を再設定することでバスを復旧させる。lastRecoverMs は、この復旧処理およびログ出力が短時間に連続して実行されることを防ぐスロットル用の時刻管理に用いられる。recoverI2C 関数をコード 6 に示す。

コード 6 AccManager::recoverI2C 関数

```
1 void AccManager::recoverI2C() {
2   unsigned long now = millis();
3   if (now - lastRecoverMs >= 1000) { // ログのスロットル
4     lastRecoverMs = now;
5   }
6
7   Wire.end();
8
9   // バスクリア: SCLを9クロック叩いて握られたSDAを解放
10  pinMode(SDA, INPUT_PULLUP);
11  pinMode(SCL, OUTPUT);
12  for (int i = 0; i < 9; i++) {
13    digitalWrite(SCL, LOW); delayMicroseconds(5);
14    digitalWrite(SCL, HIGH); delayMicroseconds(5);
15  }
16  // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
17  pinMode(SDA, OUTPUT);
18  digitalWrite(SDA, LOW); delayMicroseconds(5);
19  digitalWrite(SCL, HIGH); delayMicroseconds(5);
20  digitalWrite(SDA, HIGH); delayMicroseconds(5);
21
22  // Wire再初期化&センサ再設定
23  Wire.begin();
24  Wire.beginTransmission(ADXL345_ADDR);
25  Wire.write(REG_POWER_CTL); Wire.write(0x08);
26  Wire.endTransmission();
27  Wire.beginTransmission(ADXL345_ADDR);
28  Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
29  Wire.endTransmission();
30 }
```

次に、update 関数について説明する。この関数は loop 内で毎回呼び出され、ACC_SAMPLE_INTERVAL (20 ms) ごとに加速度の実測値を取得し、前回値との差 diff を求める。diff が検知閾値 accThreshold を超過し、かつ前回検知から shakeInterval (750 ms, 多重検知防止用) が経過している場合にのみ、振り動作として検知し updateShakedInfo 関数を呼び出す。update 関数をコード 7 に示す。

コード 7 AccManager::update 関数

```
1 void AccManager::update(SyncManager& sync, PotManager& pot) {
2   unsigned long now = millis();
3   if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
4     lastSampleTime = now;
5     currentAccVal = readAccMagnitude();
6     float diff = fabsf(currentAccVal - lastAccVal);
7
8     bool overThreshold = (diff > accThreshold);
```

```

9      bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
10
11      if (overThreshold && intervalPassed) {
12          lastDetectTime = now;
13          isStarted = !isStarted;
14          updateShakedInfo(sync, pot);
15      }
16      lastAccVal = currentAccVal;
17  }
18  }

```

最後に、updateShakedInfo 関数について説明する。この関数は、振り動作の検知をトリガとして、PotManager が保持する bpmFrag・volFrag フラグ（可変抵抗器の値が変化した場合に立つフラグ。2.1.3 項で説明する）を確認し、フラグが立っている項目についてのみ SyncManager 経由で BPM または音量の変更情報を送信する。すなわち、実際に無線送信が発生するのは、可変抵抗器を操作した後にデバイスを振った場合に限られる。updateShakedInfo 関数をコード 8 に示す。

コード 8 AccManager::updateShakedInfo 関数

```

1 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
2     if (pot.bpmFrag) {
3         sync.sendBPM(pot.getBpmStep());
4         pot.bpmFrag = false;
5     }
6     if (pot.volFrag) {
7         sync.sendVolume(pot.getVolStep());
8         pot.volFrag = false;
9     }
10 }

```

最後に、指揮デバイスにおける可変抵抗器を用いた BPM・音量読み取り機能について説明する。指揮デバイスでは、演奏の要素である BPM と音量を、ユーザが操作できるように可変抵抗器を用いて 5 段階で調整する。アナログ入力値の特性を考慮したデータ処理を行うことで、安定した制御環境を構築している。アルゴリズムは、サンプリングと移動平均化、範囲分割、パラメータの抽出と出力の 3 段階に分けられる。サンプリングと移動平均化では、SAMPLE_INTERVAL ごとに BPM および音量変更用の可変抵抗器の電圧値を、配列 bpmSamples・volSamples に sampleSize 分だけ読み取り保存する。このとき、配列の各要素は書き込み位置 (bpmReadIndex・volReadIndex) が指す最も古い値から順に上書きされ、上書きの直前に該当要素の値を合計 bpmTotal・volTotal からあらかじめ減算し、新たに読み取った値を加算することで、配列全体を読み直すことなく常に最新の合計値を保持する（移動合計）。配列が一巡し全要素が新しい値に更新された時点 (bpmBufferFull・volBufferFull が true になった時点) 以降、この合計値を sampleSize で除算した移動平均値を averageBpmVal・averageVolVal に格納する。範囲分割では、移動平均化された値を STEP_BOUNDARIES[] で定義した 4 つの閾値と比較し、1～5 の段階番号を currentBpmStep・currentVolStep に格納する。パラメータの抽出と出力では、算出した段階が直前

の段階 (prevBpmStep・prevVolStep) から変化した場合にのみ、公開フラグ bpmFrag・volFrag を立てる。外部からは getBpmStep()・getVolStep() を介して現在の段階を取得し、前述の加速度センサ制御における棒振り動作の確定時にこれらのフラグを参照して各楽器機への送信を行う。なお、音量用可変抵抗器は配線の都合上、回した方向とセンサ値の増減が逆になるため、getVolStep() は 6-currentVolStep を返すことで表示上の段階を反転させる。BPM・音量読み取り機能で使用する変数の仕様を表 7 に示す。

表 7: BPM・音量読み取り機能で使用する変数の仕様一覧

変数名	型	説明
PIN_BPM	const int	BPM 変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
PIN_VOL	const int	音量変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
SAMPLE_INTERVAL	const unsigned long	可変抵抗器の値をサンプリングする周期を定義
sampleSize	const int	移動平均算出のためのサンプル数を定義
STEP_BOUNDARIES[]	const int	段階判定に用いる 4 つの閾値を保持する配列
lastSampleTime	unsigned long	前回サンプリングを行った時間を格納
bpmSamples[]	int	BPM 用のサンプル値を保持するための配列
volSamples[]	int	音量用のサンプル値を保持するための配列
bpmReadIndex	int	bpmSamples[] 内での現在の書き込み位置を管理するインデックス
volReadIndex	int	volSamples[] 内での現在の書き込み位置を管理するインデックス
bpmBufferFull	bool	bpmSamples[] が一巡し、移動平均の算出が有効になったかを保持
volBufferFull	bool	volSamples[] が一巡し、移動平均の算出が有効になったかを保持
bpmTotal	long	bpmSamples[] 内の累積合計を格納
volTotal	long	volSamples[] 内の累積合計を格納

次ページに続く

表 7 続き

変数名	型	説明
averageBpmVal	float	算出された BPM 用のアナログ入力の移動平均値を格納
averageVolVal	float	算出された音量用のアナログ入力の移動平均値を格納
currentBpmStep	uint8_t	算出された BPM の 5 段階のレベル (1～5) を格納
currentVolStep	uint8_t	算出された音量の 5 段階のレベル (1～5) を格納
prevBpmStep	int	段階の変化検知のための直前の BPM 段階を格納
prevVolStep	int	段階の変化検知のための直前の音量段階を格納
bpmFrag	bool	BPM 段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用
volFrag	bool	音量段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用

続いて、可変抵抗器読み取り機能（PotManager クラス）で使用する関数の仕様を表 8 に示す。

表 8 可変抵抗器読み取り機能 (PotManager クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
update	void	なし	SAMPLE_INTERVAL ご と に BPM・音量それぞれの可変 抵抗器の値をサンプリングし、 移動合計・移動平均・段階算 出を行う。loop 内で毎回呼び 出す
calcStep	int	float val	0～1023 の 移 動 平 均 値 を STEP_BOUNDARIES[] と 比 較 し、1～5 のいずれかの段階を 返す
bpmUpdatePotentiometers	bool	なし	BPM 用可変抵抗器の値を読み 取り移動合計・移動平均を更 新し、段階が変化した場合に true を返す
volUpdatePotentiometers	bool	なし	音量用可変抵抗器の値を読み 取り移動合計・移動平均を更 新し、段階が変化した場合に true を返す
getBpmStep	uint8_t	なし	現在の BPM 段階 (currentBpm- Step) を外部に返すゲッター
getVolStep	uint8_t	なし	現在の 音量 段階を外部に 返すゲッター。配線特性に よる値の反転を補正し、6- currentVolStep を返す

以下では、表 8 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、update 関数について説明する。この関数は loop 内で毎回呼び出され、SAMPLE_INTERVAL (20 μ s) ご と に bpmUpdatePotentiometers 関数と volUpdatePotentiometers 関数を呼び出す。update 関数をコード 9 に示す。

コード 9 PotManager::update 関数

```

1 void PotManager::update() {
2     unsigned long now = micros();
3     if (now - lastSampleTime >= SAMPLE_INTERVAL) {
4         lastSampleTime = now;
5         if (bpmUpdatePotentiometers()) bpmFrag = true;
6         if (volUpdatePotentiometers()) volFrag = true;
    }
}

```

```

7   }
8   }

```

次に、bpmUpdatePotentiometers 関数について説明する。この関数は、リングバッファ（配列 bpmSamples, 添字 bpmReadIndex）を用いた移動平均処理により、analogRead で取得したアナログ値のノイズを平滑化する。具体的には、配列の最も古いサンプルを合計値 bpmTotal から減算し、新たに取得した値を加算したうえで書き込み位置を進める。sampleSize（10 サンプル）分のバッファが一巡した時点から、合計値をサンプル数で割った移動平均 averageBpmVal を算出し、calcStep 関数で段階へ変換したうえで、前回段階から変化していれば true を返す。bpmUpdatePotentiometers 関数をコード 10 に示す。

コード 10 PotManager::bpmUpdatePotentiometers 関数

```

1  bool PotManager::bpmUpdatePotentiometers() {
2      bpmTotal -= bpmSamples[bpmReadIndex];
3      bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
4      bpmTotal += bpmSamples[bpmReadIndex];
5      bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
6      if (bpmReadIndex == 0) bpmBufferFull = true;
7      if (!bpmBufferFull) return false;
8
9      averageBpmVal = (float)bpmTotal / sampleSize;
10     currentBpmStep = calcStep(averageBpmVal);
11
12     if (currentBpmStep != prevBpmStep) {
13         prevBpmStep = currentBpmStep;
14         return true;
15     }
16     return false;
17 }

```

続いて、calcStep 関数について説明する。この関数は、bpmUpdatePotentiometers 関数および volUpdatePotentiometers 関数から呼び出され、得られた移動平均値を 1～5 の 5 段階に離散化する。STEP_BOUNDARIES 配列（300, 500, 650, 818）に対する大小比較によって段階を決定しており、境界値の間隔を均等にしていない理由は、可変抵抗器の実測特性に合わせて各段階の操作しやすさを調整しているためである。calcStep 関数をコード 11 に示す。

コード 11 PotManager::calcStep 関数

```

1  int PotManager::calcStep(float val) {
2      if (val <= STEP_BOUNDARIES[0]) return 1;
3      else if (val <= STEP_BOUNDARIES[1]) return 2;
4      else if (val <= STEP_BOUNDARIES[2]) return 3;
5      else if (val <= STEP_BOUNDARIES[3]) return 4;
6      else return 5;
7  }

```

次に、volUpdatePotentiometers 関数について説明する。この関数は bpmUpdatePotentiometers 関数と同様の移動平均処理を行うが、対象のピンおよび内部変数が音量用（PIN_VOL, volSamples

等)に置き換わっている点のみが異なり、処理の骨格は共通している。volUpdatePotentiometers 関数をコード 12 に示す。

コード 12 PotManager::volUpdatePotentiometers 関数

```
1 bool PotManager::volUpdatePotentiometers() {
2     volTotal -= volSamples[volReadIndex];
3     volSamples[volReadIndex] = analogRead(PIN_VOL);
4     volTotal += volSamples[volReadIndex];
5     volReadIndex = (volReadIndex + 1) % sampleSize;
6     if (volReadIndex == 0) volBufferFull = true;
7     if (!volBufferFull) return false;
8
9     averageVolVal = (float)volTotal / sampleSize;
10    currentVolStep = calcStep(averageVolVal);
11
12    if (currentVolStep != prevVolStep) {
13        prevVolStep = currentVolStep;
14        return true;
15    }
16    return false;
17 }
```

最後に、getBpmStep 関数および getVolStep 関数について説明する。いずれも PotManager.h 内でインライン定義されたゲッター関数であり、外部 (AccManager クラス) から現在の段階番号を取得するために用いられる。getVolStep 関数では、可変抵抗器を回転させる向きと、音量として直感的に理解しやすい段階の増減方向を一致させるため、6 - currentVolStep という変換を行っている点 getBpmStep 関数と異なる。両関数をコード 13 に示す。

コード 13 PotManager::getBpmStep 関数・getVolStep 関数

```
1 uint8_t getBpmStep() const { return currentBpmStep; }
2 uint8_t getVolStep() const { return 6 - currentVolStep; }
```

なお、本節で説明した SyncManager クラスによって送信された演奏開始・終了、BPM、音量の各情報が、中継機デバイス側でどのように受信・反映されるかについては 2.2.3 項 (中継機デバイスの内部設計) で説明する。

2.2 中継機デバイス

2.2.1 概要

次に、中継機デバイスについて説明する。中継機デバイスの構成図を図 9 に示す。本デバイスは、指揮デバイスからの情報を受け取り中継機デバイスの回転を作動させ、現在の BPM を表示するデバイスであり、主に以下の 2 つの機能を有する。

第一の機能は、上記で述べた BPM に合わせて回転し楽曲の BPM を制御する機能である。デバイスに設置してあるレーザー受光のテンポで BPM を管理し、楽曲のテンポを視覚的にも理解できる

ようにする。

第二の機能は、Arduino Uno R4 WiFi に付属している LEDMatrix に BPM を表示する機能である。ここに指揮デバイスから受け取った BPM を表示し、視覚的に現在流れている楽曲の BPM を理解できるようにする。ここからは、外部設計と内部設計について説明する。

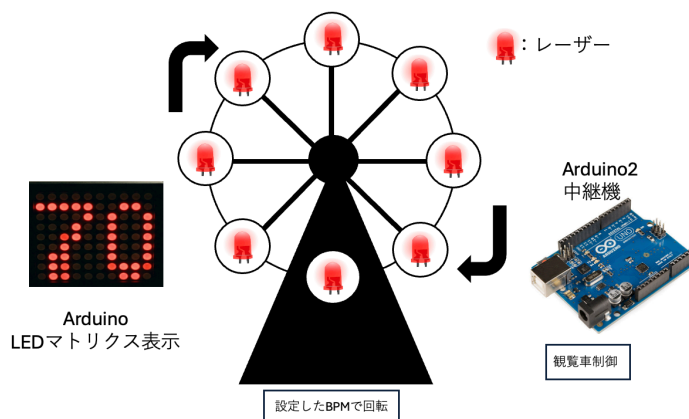


図 9 中継機デバイスの構成図

2.2.2 外部設計

本節では中継機デバイスの外部設計について説明する。まず、本デバイスを構成する部分について表 9 に示す。この内、個別で購入する必要があるものはハイパワーギヤーボックス、ブレッドボード用 DC ジャック DIP 化キット、ボタン電池、ボタン電池ホルダー、QI ケーブル、有極コンデンサーである。

また、中継機の回路図を図 10 に示す。モータードライバーに Arduino D5, D6 ピンを接続し、PWM 制御を行えるようにする。モータードライバーへの電力供給は当初、外部電源として 9V のアルカリ電池を検討していたが、Arduino の 5V からの電力供給と DC ジャックからの AC アダプター 5V1A による外部給電により、アルカリ電池が不要であることが実装時に確認されたため除外した。また、モータードライバー及び DC ジャックはピンヘッダーのはんだ付けを行う。加えて、ギアボックスは $0.01\mu\text{F}$ のコンデンサーと直接はんだ付けを行う。ギアボックスと並列に接続された $0.01\mu\text{F}$ のコンデンサは、モータのブラシ接点で発生する電気ノイズを吸収し、制御信号や周辺回路への影響を低減する役割を果たしている。

次に、観覧車の設計について説明する。作成するために、3D プリンタを用いて作成する。実際に設計に使用するモデルデータを図 11 から図 16 に示している。観覧車の直径は約 16cm とし、ボタン電池付きレーザーを 45 度間隔で配置する。そして、観覧車を支える左右の支柱および土台を設計する。観覧車本体は、以下のパーツに分割して設計を行う。

- ・土台 観覧車全体を支える部分であり、内部を空洞化させる。また、支柱を土台に差し込み、固定する役割も持つ。実際に設計する部品は図 11 の通りである。

表 9 中継機デバイスを構成する部品

機材	品番	個数 [個]
Arduino Uno R4 WiFi	-	1
ルーター	-	1
ブレッドボード	-	1
赤色ドットレーザーモジュール	FU650AD5-C6	8
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4
ボタン電池	CR2477	4
スライドスイッチ	SS-12D00-G5	4
モータードライバーモジュール	AE-DRV8835-S	1
ハイパワーギヤーボックス HE	72003	1
ジャンパーワイヤー	-	適宜
ブレッドボード用 2.1mm 標準 DC ジャック DIP 化キット	AE-DC-POWER-JACK- DIP	1
超小型スイッチング AC アダプター 5 V 1 A	-	1
QI ケーブル 2P-1Px2	311-262	1
抵抗器 10 Ω	-	1
コンデンサー 0.01 μF	-	1
コンデンサー 0.047 μF	-	1
有極性コンデンサー 100 μF	-	1

- 支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 12 の通りである。
- 回転軸 ギヤボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるよう、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 13 の通りである。
- 回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は図 14, 15 の通りである。
- 受光部 レーザーの延長線上に設置する照度センサーの受光部である。回転リングから平行な位置に円盤を設置し、中央にギアボックスモータを設置する台を設ける。90 度おきに円盤にくぼみを作り、センサーを設置できるようにした。受光部の円盤によりレーザー光が後ろに漏

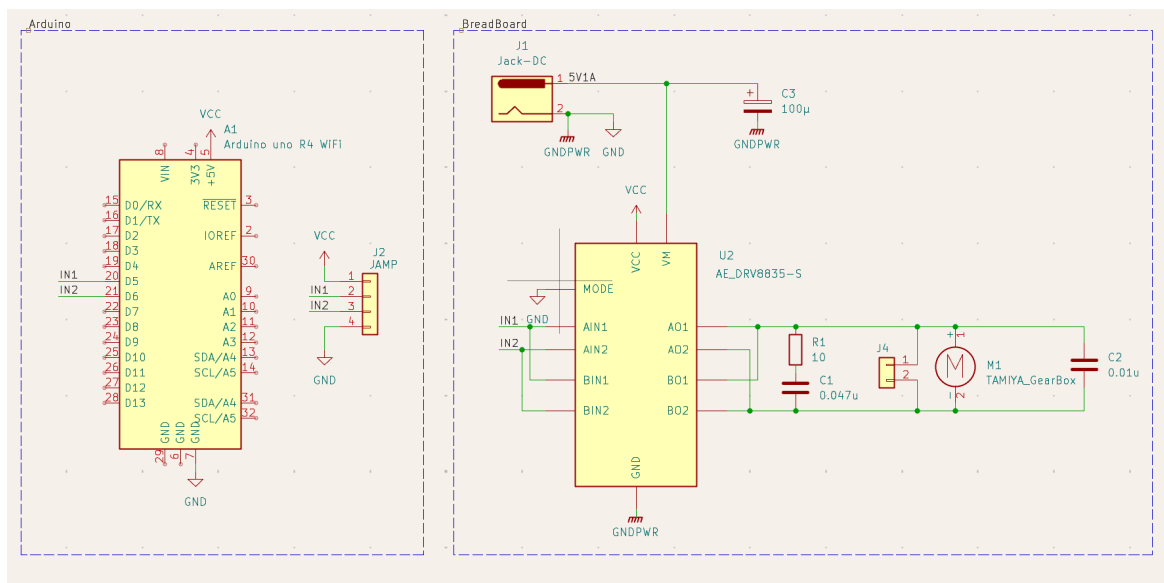


図 10 観覧車型中継機デバイス回路図

れないという設計となっている。実際に設計する部品は図 16 の通りである。

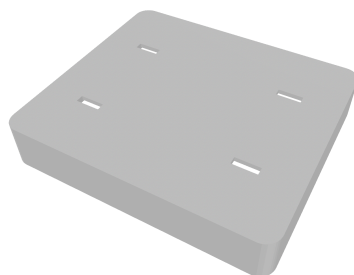


図 11 設計に使用するモデルデータ (土台部)

図～～～に実際に作成した中継機デバイスを示す。

2.2.3 内部設計

本節では中継機デバイスの内部設計について説明する。主に回転速度同期機構（モータ制御）と LEDMatrix 表示機構のプログラムについて説明する。

次に、中継機デバイスの制御プログラムのファイル構成を示す。表 10 の通り、各ファイルは機能ごとに分割されている。

FerrisWheel.ino の loop 関数は、Udp.parsePacket() でパケットの到着を監視し、届いていれば uint8_t packetBuffer[10] として 2 バイト分を読み取る。先頭バイト packetBuffer[0] をヘッダ文字として判定し、ヘッダが 'S'（演奏開始）の場合は motor.startRamp() と displayBPM() を、'B'（BPM 変更）の場合は motor.changeBPM() と displayBPM() を、'E'（演奏終了）の場合は motor.stopRamp() と

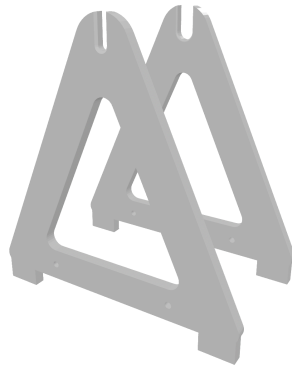


図 12 設計に使用するモデルデータ (支柱部)



図 13 設計に使用するモデルデータ (回転軸部)

clearDisplay() を、2 バイト目のペイロード（BPM 段階番号）を引数として呼び出す構成となっている。loop 関数をコード 14 に示す。

コード 14 FerrisWheel.ino の loop 関数

```

1 void loop() {
2     uint8_t packetSize = Udp.parsePacket();
3     if (packetSize <= 0) return;
4     if (packetSize != 2) {
5         while (Udp.available()) Udp.read();
6         return;
7     }
8
9     Udp.read(packetBuffer, sizeof(packetBuffer));
10    char header = packetBuffer[0];
11    uint8_t payload = packetBuffer[1];
12
13    switch (header) {
14        case 'S':
15            running = true;
16            motor.startRamp(payload);
17            displayBPM(payload);
18            break;
19        case 'B':
20            if (!running) break;
21            motor.changeBPM(payload);
22            displayBPM(payload);
23            break;

```

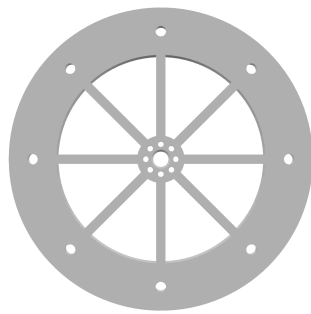


図 14 設計に使用するモデルデータ (回転リング部 1)

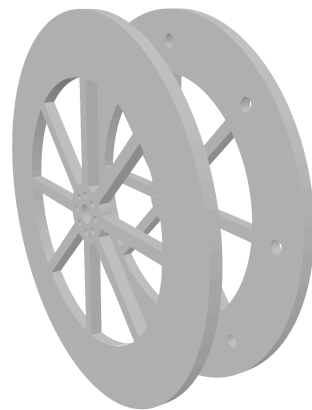


図 15 設計に使用するモデルデータ (回転リング部 2)

```

24     case 'E':
25         running = false;
26         motor.stopRamp();
27         clearDisplay();
28         break;
29     }
30 }
```

続いて、Main (FerrisWheel.ino) で使用する変数の仕様を表 11 に示す。

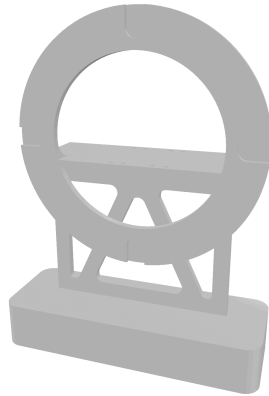


図 16 設計に使用するモデルデータ (受光部)

表 10 中継機デバイスのファイル構成

ファイル名	概要
FerrisWheel.ino	WiFi 接続・UDP 受信の管理, 受信ヘッダに応じたモータ制御・表示処理の呼び出し
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理
Display.h	定数定義, 関数宣言, 外部参照
MotorControl.cpp	PWM 出力によるモータ回転制御, 回転速度テーブルの生成
MotorControl.h	モータ制御用の定数・変数定義, 関数宣言

表 11 Main (FerrisWheel.ino) で使用する変数の仕様一覧

変数名	型	説明
SSID	const char[]	Wi-Fi のネットワーク名 (hackathon001-WPA2) を定義
PASS	const char[]	その Wi-Fi のパスワード Wi-Fi パスワードを定義
localIP	IPAddress	中継機デバイス自身の静的 IP アドレスを定義
gateway	IPAddress	静的 IP 設定に使用するデフォルトゲートウェイのアドレスを定義
subnet	IPAddress	静的 IP 設定に使用するサブネットマスクを定義
LOCAL_PORT	const uint16_t	UDP 受信に利用する自身のローカルポート番号を定義
packetBuffer	uint8_t[10]	UDP 通信で受信したパケットのデータを一時的に格納するバッファ配列
running	bool	演奏中かどうかを判定し, 演奏前 (待機中) の予期せぬモータ回転を防ぐための状態管理フラグ

続いて、Main (FerrisWheel.ino) で使用する関数の仕様を表 12 に示す。

表 12 Main (FerrisWheel.ino) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
loop	void	なし	UDP パケットの受信確認を行い、ヘッダが'S'であれば motor.startRamp() と displayBPM() を、'B' であれば motor.changeBPM() と displayBPM() を、'E' であれば motor.stopRamp() と clearDisplay() を呼び出す。演奏前 (running=false) に'B'を受信した場合は無視する。

ここからは、モータの制御、LEDMatrix での BPM 表示、回転速度実測プログラム (LaserIntervalTest) の順に、使用する変数・関数の仕様と処理内容について説明する。

まず、中継機デバイスにおけるモータの制御処理について説明する。本デバイスのギアボックスモータの回転速度を制御するために、Arduino の PWM(パルス幅変調) 機能を用いてモータの平均印加電圧を制御する。PWM とは、デジタル出力ピンから出力される矩形波の HIGH 状態と LOW 状態の時間比率 (デューティ比) を変化させることで、擬似的にアナログ電圧を生成する制御方式である。Arduino のデジタル出力ピンは 5V の HIGH か 0V の LOW のいずれかしか出力できないが、HIGH と LOW を高速に切り替えてデューティ比を調整することで、負荷に対して見かけ上の連続的な電圧を印加することが可能となる。

本デバイスは、Arduino の D5 ピンをモータドライバーモジュールに接続し、PWM 制御を行う構成としている。Arduino の analogWrite 関数は、引数として 0 から 255 の整数値を受け取り、この値に応じたデューティ比の矩形波を出力する。このとき引数が 0 のときデューティ比は常時 LOW であり、255 のときデューティ比は常時 HIGH となる。中間値では、引数に比例したデューティ比の矩形波が生成される。この関係を式で表すと、供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (1)$$

このとき、モータの巻線が持つインダクタンスとギアボックスの機械的慣性が電氣的・機械的なローパスフィルタとして機能するため、モータはパルスの平均値に応じた一定速度で滑らかに回転する。

次に、ルックアップテーブルによる速度制御について説明する。一般的な DC モータの回転速度は供給電圧に比例する性質があるが [11]、実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い。そのため、当初検討していた「4 拍で 90 度 (1 小節が 90 度) 回転する」といった幾何学的な理論式による制御ではなく、より確実な速度追従を実現するため、幾何学的な縛りを廃止して実測値に基づくルックアップテーブル方式を採用する。

本デバイスでは、表 13 に示す離散的な 5 段階の BPM 設定値と、あらかじめ実測により求めた PWM 出力値を、コード内の配列 bpmTable・pwmTable として対応させ、受信した段階番号に応じた pwmValue を即座に選択・出力する構成とする。表 13 に示す実測周期は、後述する回転速度実測プログラム（LaserIntervalTest）を用いて計測した値に基づいて決定している。

表 13 BPM に対する実測周期と出力 PWM 値			
段階	目標 BPM	実測周期 [ms]	出力 PWM 値
1	60	230	26
2	90	180	28
3	120	140	30
4	150	110	32
5	180	80	37

続いて、モータ制御（MotorControl クラス）で使用する変数・定数の仕様を表 14 に示す。

表 14 モータ制御（MotorControl クラス）で使用する変数・定数の仕様一覧		
変数名	型	説明
PIN_MOTOR_PWM	const uint8_t	モータ制御用の出力ピンを定義
bpmTable[5]	const uint8_t	5 つの離散値 BPM を格納
pwmTable[5]	const uint8_t	5 つの離散値 PWM 出力値を格納
VCC	const float	Arduino への供給電圧 V_{cc} を定義
currentBPM	uint8_t	現在演奏中の BPM データを格納
RPM	float	BPM から計算された目標回転数を格納
omega	float	目標角速度 ω を格納
pwmValue	uint8_t	analogWrite 用の出力値を格納
V_average	float	平均電圧 $V_{average}$ を格納
rpmTable[5]	float	bpmTable に対応する目標回転数を格納する配列

続いて、モータ制御（MotorControl クラス）で使用する関数の仕様を表 15 に示す。

表 15 モータ制御 (MotorControl クラス) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
begin	void	なし	setup 内で一度だけ呼び出される関数であり、モータピンの初期化を行う
createRpmTable	void	なし	bpmTable の各値から目標回転数を算出し、rpmTable を作成する
startRamp	void	uint8_t stepNumber	演奏開始時の加速を制御する。段階番号から求めた目標の pwmValue まで、PWM 出力値を 5 ずつ段階的に上げていくことで滑らかに目標速度へ到達させる
stopRamp	void	なし	演奏終了時の減速を制御する。現在の pwmValue から 0 まで、PWM 出力値を 5 ずつ段階的に下げていくことで滑らかに停止させる
changeBPM	void	uint8_t stepNumber	演奏中の BPM 変更を行う。startRamp とは異なり、段階的な加減速を行わず目標の pwmValue を即座に出力する
stepToNumber	void	uint8_t stepNumber	受信した段階番号 (1~5) から、bpmTable・pwmTable・rpmTable を参照して currentBPM・pwmValue・RPM を求める
updateStatus	void	なし	RPM から角速度 omega を、pwmValue から平均電圧 V_average を算出する

以下では、表 15 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、begin 関数について説明する。この関数は、setup 内で一度だけ呼び出され、モータドライバへ接続された PIN_MOTOR_PWM を出力ピンとして初期化したうえで、出力を 0 にリセットする。begin 関数をコード 15 に示す。

コード 15 MotorControl::begin 関数

```

1 void MotorControl::begin() {
2     pinMode(PIN_MOTOR_PWM, OUTPUT);
3     analogWrite(PIN_MOTOR_PWM, 0);
4 }

```

次に、createRpmTable 関数について説明する。この関数も setup 内で一度だけ呼び出され、bpmTable の各要素を 16 で除算することで rpmTable を生成する。createRpmTable 関数をコード 16 に示す。

コード 16 MotorControl::createRpmTable 関数

```
1 void MotorControl::createRpmTable() {
2     for (uint8_t i = 0; i < 5; i++) {
3         rpmTable[i] = bpmTable[i] / 16.0f;
4     }
5 }
```

次に、startRamp 関数について説明する。この関数は、まず stepToNumber 関数によって目標の pwmValue を求めたうえで、出力値を 0 から pwmValue まで 5 刻みで増加させながら 40 ms ずつ待機することで、モータを滑らかに加速させる。startRamp 関数をコード 17 に示す。

コード 17 MotorControl::startRamp 関数

```
1 void MotorControl::startRamp(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     for (int i = 0; i <= pwmValue; i += 5) {
4         analogWrite(PIN_MOTOR_PWM, i);
5         delay(40);
6     }
7     analogWrite(PIN_MOTOR_PWM, pwmValue);
8 }
```

次に、stopRamp 関数について説明する。この関数は startRamp 関数と対になる処理であり、現在の pwmValue から 0 まで 5 刻みで減少させながら 40 ms ずつ待機することで、モータを滑らかに減速・停止させる。stopRamp 関数をコード 18 に示す。

コード 18 MotorControl::stopRamp 関数

```
1 void MotorControl::stopRamp() {
2     for (int i = pwmValue; i >= 0; i -= 5) {
3         analogWrite(PIN_MOTOR_PWM, i);
4         delay(40);
5     }
6     analogWrite(PIN_MOTOR_PWM, 0);
7 }
```

次に、changeBPM 関数について説明する。この関数は演奏中に BPM の段階が変更された際に呼び出され、stepToNumber 関数と updateStatus 関数によって新しい段階の値を求めたうえで、startRamp 関数のような段階的な加減速を行わず、目標の pwmValue を即座に出力する。演奏を止めずにテンポを追従させるため、段階的な加減速は行わない仕様としている。changeBPM 関数をコード 19 に示す。

コード 19 MotorControl::changeBPM 関数

```
1 void MotorControl::changeBPM(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     updateStatus();
4     analogWrite(PIN_MOTOR_PWM, pwmValue);
5 }
```

最後に、stepToNumber 関数と updateStatus 関数について説明する。stepToNumber 関数は、受信

した段階番号(1~5)が範囲内であることを確認したうえで、bpmTable・pwmTable・rpmTableの該当インデックスから currentBPM・pwmValue・RPM をそれぞれ求める。updateStatus 関数は、この RPM および pwmValue を基に、(1) 式に相当する計算によって omega と V_average を算出する。両関数をコード 20 に示す。

コード 20 MotorControl::stepToNumber 関数・updateStatus 関数

```

1 void MotorControl::stepToNumber(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     currentBPM = bpmTable[stepNumber - 1];
4     pwmValue   = pwmTable[stepNumber - 1];
5     RPM        = rpmTable[stepNumber - 1];
6 }
7
8 void MotorControl::updateStatus() {
9     omega      = 2.0 * PI * RPM / 60.0f;
10    V_average = VCC * (pwmValue / 256.0f);
11 }

```

次に、中継機デバイスにおける LEDMatrix 表示処理について説明する。今回のシステムでは表示用のマトリクスとして、Arduino Uno R4 WiFi 付属のマトリクスを使用する。このマトリクスを用いて、FerrisWheel.ino の loop 関数から受信した BPM 段階番号を数値変換し、独自定義したフォント配列を用いて表示を行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 のドットパターンをコード 21 に示す。

コード 21 font3x5 配列における数字 0 のドットパターン定義

```

1 {1,1,1,
2   1,0,1,
3   1,0,1,
4   1,0,1,
5   1,1,1}

```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3 × 5 のフォント配列を用いてフレームバッファに書き込むことで行う。続いて、LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様を表 16 に示す。

表 16 LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様一覧

変数名	型	説明
BPM_STEP_1~5	#define	BPM5 段階 (60,90,120,150,180) の各値を定義
FONT_WIDTH	#define	1 文字の横ドット数 (3) を定義
FONT_HEIGHT	#define	1 文字の縦ドット数 (5) を定義
MATRIX_ROWS	#define	LED マトリクスの行数 (8) を定義
MATRIX_COLS	#define	LED マトリクスの列数 (12) を定義
font3x5[10][15]	const uint8_t	独自定義した 0~9 の 3 × 5 ドットパターンフォント配列
matrix	ArduinoLEDMatrix	LED マトリクスのインスタンス
bpmTable[5]	static const int	段階番号 (1~5) から実際の BPM 値へ変換するための対応表
frameBuffer[8][12]	static uint8_t	描画用の共有フレームバッファ
currentBPM	static int	現在表示中の BPM 値を保持する内部状態変数. clearDisplay 呼び出し時に初期値 (-1) へリセットされる

続いて, LEDMatrix 表示 (Display モジュール) で使用する関数の仕様を表 17 に示す.

表 17 LEDMatrix 表示 (Display モジュール) で使用する関数の仕様一覧

関数名	戻り値	引数	説明
displaySetup	void	なし	LED マトリクスの初期化を行う
displayBPM	void	uint8_t stepNumber	受信した BPM 段階番号を LED マトリクスに数値表示する
drawDigit	void	int digit, int x_offset	1 桁の数字をフレームバッファの指定位置に描画する
clearDisplay	void	なし	LED マトリクスを全消灯する

LEDMatrix プログラムのクラス図は以下の図 17 の通りである. WiFi 接続・UDP 受信および各処理の呼び出しを行う FerrisWheel.ino, LED マトリクスへの描画処理を行う Display.cpp, および関数宣言やフォント定義を行う Display.h で構成される. 各モジュール間の関係について, FerrisWheel.ino から Display.cpp への関係は, 表示処理を利用する依存関係である. また, Display.cpp から Display.h への関係は, 関数宣言およびフォント定義を参照する依存関係である.

以下では, 表 17 に示した順に, 各関数の処理内容を実際のコードとともに説明する. はじめに, displaySetup 関数について説明する. この関数は, matrix.begin() を呼び出した後, 消灯フレームを一度描画してから 200 ms 待機する. これは, begin() 直後の最初の loadFrame() は反映されない場合があるための対策である. displaySetup 関数をコード 22 に示す.

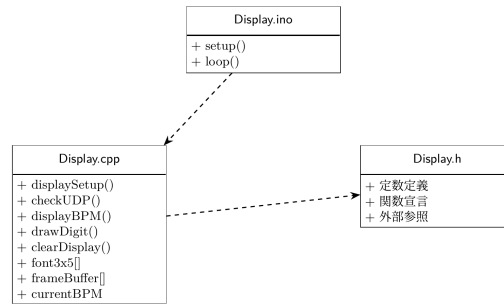


図 17 LEDMatrix のクラス図

コード 22 displaySetup 関数

```

1 void displaySetup() {
2     matrix.begin();
3     uint32_t warmUp[3] = {0, 0, 0};
4     matrix.loadFrame(warmUp);
5     delay(200);
6 }
  
```

次に、displayBPM 関数について説明する。この関数は、loop 関数から受け取った BPM 段階番号（1～5）を bpmTable[] で参照して実際の BPM 値に変換したうえで各桁ごとに分割し、drawDigit() を用いてフレームバッファに描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。displayBPM 関数をコード 23 に示す。

コード 23 displayBPM 関数

```

1 void displayBPM(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     uint8_t bpm = bpmTable[stepNumber - 1];
4
5     memset(frameBuffer, 0, sizeof(frameBuffer));
6
7     int digits[3];
8     int numDigits;
9     if (bpm >= 100) {
10         numDigits = 3;
11         digits[0] = bpm / 100;
12         digits[1] = (bpm / 10) % 10;
13         digits[2] = bpm % 10;
14     } else if (bpm >= 10) {
15         numDigits = 2;
16         digits[0] = bpm / 10;
17         digits[1] = bpm % 10;
18     } else {
19         numDigits = 1;
20         digits[0] = bpm;
21     }
22 }
  
```

```

23     int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
24     int startX     = (MATRIX_COLS - totalWidth) / 2;
25
26     for (int i = 0; i < numDigits; i++) {
27         drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
28     }
29
30     uint32_t bitmap[3] = {0, 0, 0};
31     for (int row = 0; row < MATRIX_ROWS; row++) {
32         for (int col = 0; col < MATRIX_COLS; col++) {
33             if (frameBuffer[row][col]) {
34                 int n = row * MATRIX_COLS + col;
35                 bitmap[n / 32] |= (1UL << (31 - (n % 32)));
36             }
37         }
38     }
39     matrix.loadFrame(bitmap);
40 }

```

次に、drawDigit 関数について説明する。この関数は、引数の数字が 0 から 9 の範囲内であるかを確認した後、縦方向の中央寄せオフセットを算出し、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取ってフレームバッファの指定位置に書き込む。drawDigit 関数をコード 24 に示す。

コード 24 drawDigit 関数

```

1 void drawDigit(int digit, int x_offset) {
2     if (digit < 0 || digit > 9) return;
3     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
4     for (int y = 0; y < FONT_HEIGHT; y++) {
5         for (int x = 0; x < FONT_WIDTH; x++) {
6             int col = x_offset + x;
7             int row = y_offset + y;
8             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
9                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
10            }
11        }
12    }
13 }

```

最後に、clearDisplay 関数について説明する。この関数は、BPM 値を初期値 (-1) にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。clearDisplay 関数をコード 25 に示す。

コード 25 clearDisplay 関数

```

1 void clearDisplay() {
2     currentBPM = -1;
3     uint32_t blank[3] = {0, 0, 0};
4     matrix.loadFrame(blank);

```

最後に、中継機デバイスのモータ回転速度を実測するために用いたプログラムである `LaserIntervalTest` について説明する。このプログラムは中継機デバイス本体には搭載せず、開発段階で観覧車に設置したレーザーと CdS セルを用いて、各 PWM 出力値における実際の回転周期を計測するために使用したものであり、表 13 に示した実測周期の値はこのプログラムによって得られている。本プログラムは、CdS セルの出力値を監視し、レーザー光の通過（パルス）を検知するたびに、直前のパルス検知からの経過時間（interval）をシリアルモニタへ出力する。周囲光量やレーザーの取り付け誤差による検知レベルのばらつきに対応するため、固定閾値ではなく、起動直後のキャリブレーションと直近のピーク値履歴に基づいて検知しきい値を継続的に調整する適応しきい値方式を採用している。`LaserIntervalTest` で使用する主な定数の仕様を表 18 に示す。

表 18 `LaserIntervalTest` で使用する主な定数の仕様一覧

定数名	型		説明
<code>CDS_PIN</code>	<code>const int</code>		CdS セルの出力を読み取るアナログピン番号を定義
<code>CDS_ON_THRESHOLD</code> <code>CDS_OFF_THRESHOLD</code>	/	<code>const int</code>	起動直後にのみ使用する仮の ON/OFF しきい値を定義
<code>CDS_MIN_DELTA</code>	<code>const int</code>		暗い状態との差がこの値未満であればレーザーとして扱わない閾値を定義
<code>CDS_PEAK_HISTORY</code>	<code>const uint8_t</code>		しきい値の再計算に用いる直近ピーク値の保持個数を定義
<code>CDS_ON_RATIO</code> / <code>CDS_OFF_RATIO</code>	<code>const uint8_t</code>		baseline からピーク値までの何%を ON/OFF しきい値とするかを定義
<code>CDS_THRESHOLD_STEP_LIMIT</code>	<code>const int</code>		1 回の再計算でしきい値が変化できる上限幅を定義
<code>CDS_STARTUP_CALIBRATION_MS</code>	<code>const long</code>	<code>unsigned</code>	起動直後の誤検出を防止するキャリブレーション時間を定義
<code>DEBOUNCE_MS</code>	<code>const long</code>	<code>unsigned</code>	同一パルスの多重検知を防止するデバウンス時間を定義

続いて、`LaserIntervalTest` (`LaserDetector` クラス) で使用する主な関数の仕様を表 19 に示す。

表 19 LaserIntervalTest (LaserDetector クラス) で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
update	bool	int cdsValue, unsigned long nowMs, unsigned long &intervalOut	loop 内で毎回呼び出される公開関数. detectLight による検知およびデバウンス処理を経て, 新規パルスを検知した場合に true を返し, 前回検知からの経過時間 intervalOut を書き込む
detectLight	bool	int v, unsigned long nowMs	baseline の追従, 起動直後のキャリブレーション, ヒステリシスを持った ON/OFF 判定によってレーザーパルスの立ち上がりを検知する内部関数
registerPeak	void	int peak	検知したパルスのピーク値を履歴に登録し, その平均値から ON/OFF しきい値を再計算する内部関数
updateFallbackThresholds	void	なし	baseline の値から ON/OFF しきい値の暫定値を算出する内部関数
limitStep	int	int current, int target, int limit	しきい値の 1 回あたりの変化量を limit 以内に制限する内部関数

以下では, 表 19 に示した主要な関数のうち, 公開インタフェースである update 関数と, 検知処

理の中心である detectLight 関数について、実際のコードとともに説明する。なお、registerPeak 関数・updateFallbackThresholds 関数・limitStep 関数の詳細な実装は付録に示すプログラム全文を参照されたい。

はじめに、setup 関数と loop 関数について説明する。setup 関数ではシリアル通信を開始するのみであり、loop 関数では CdS セルの値を毎回取得し、LaserDetector の update 関数へ渡している。新しいパルスが検知された場合は、得られた interval[ms] をシリアルモニタへ出力する。setup 関数・loop 関数をコード 26 に示す。

コード 26 LaserIntervalTest.ino の setup 関数・loop 関数

```
1 void setup() {
2   Serial.begin(115200);
3   Serial.println("---レーザー間隔計測_Setup_Started_---");
4 }
5
6 void loop() {
7   unsigned long now = millis();
8   int cdsValue = analogRead(CDS_PIN);
9   unsigned long interval = 0;
10
11   if (detector.update(cdsValue, now, interval)) {
12     Serial.print("[interval]_");
13     Serial.print(interval);
14     Serial.println("_ms");
15   }
16 }
```

次に、LaserDetector クラスの update 関数について説明する。この関数は、detectLight 関数によってパルスの立ち上がりが検知され、かつ前回検知から DEBOUNCE_MS (200 ms) 以上が経過している場合にのみ、true を返して interval (前回検知時刻との差) を書き込む。update 関数をコード 27 に示す。

コード 27 LaserDetector::update 関数

```
1 bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
2   if (calibrationStartMs == 0) calibrationStartMs = nowMs;
3   if (!detectLight(cdsValue, nowMs)) return false;
4   if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
5
6   intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
7   bool hasInterval = (lastDetectTime != 0);
8   lastDetectTime = nowMs;
9   return hasInterval;
10 }
```

最後に、detectLight 関数について説明する。この関数は、暗状態の推定値である baseline を継続的に追従させながら、値が baseline+CDS_MIN_DELTA 相当の onThreshold を上回った瞬間をパルスの立ち上がりとして検知する状態機械である。起動直後は startupCalibrated フラグが false の間、

CDS_STARTUP_CALIBRATION_MS (1 秒) をかけて最も暗い値を baseline の初期値として探索し、誤検知を防止する。また、キャリブレーション完了時点でレーザーが既に照射状態であった場合は、waitingForRelease フラグによって一度非照射状態に戻るまで検知を保留する。パルスの立ち下がり (照射終了) を検知すると registerPeak 関数を呼び出し、しきい値を実測値に基づいて更新する。detectLight 関数をコード 28 に示す。

コード 28 LaserDetector::detectLight 関数

```
1 bool detectLight(int v, unsigned long nowMs) {
2     if (!baselineReady) {
3         baseline = v;
4         startupMin = v;
5         baselineReady = true;
6         updateFallbackThresholds();
7     }
8
9     if (!startupCalibrated) {
10        if (v < startupMin) startupMin = v;
11        baseline = startupMin;
12        updateFallbackThresholds();
13        if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
14        startupCalibrated = true;
15        waitingForRelease = (v >= onThreshold);
16        releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
17                           : offThreshold;
18        return false;
19    }
20
21    if (waitingForRelease) {
22        if (v <= releaseThreshold) {
23            baseline = v;
24            updateFallbackThresholds();
25            waitingForRelease = false;
26        }
27        return false;
28    }
29
30    if (!laserOn) {
31        if (v < baseline) baseline = v;
32        else baseline = (baseline * 31 + v) / 32;
33        if (!calibrated) updateFallbackThresholds();
34    }
35
36    if (!laserOn && v >= onThreshold) {
37        laserOn = true;
38        pulsePeak = v;
39        return true;
40    }
```

```

40
41     if (laserOn) {
42         if (v > pulsePeak) pulsePeak = v;
43         if (v <= offThreshold) {
44             laserOn = false;
45             registerPeak(pulsePeak);
46         }
47     }
48     return false;
49 }

```

プログラム全文は付録に示す。

なお、本節で説明した FerrisWheel.ino の UDP 受信処理は、指揮デバイスの SyncManager クラスから送信された演奏開始・終了、BPM、音量の各情報を受信する側の処理であり、送信側の詳細は 2.1.3 項（指揮デバイスの内部設計）を参照されたい。また、中継機デバイスから楽器デバイスへの同期は、回転する観覧車に設置されたレーザーの通過光を各楽器デバイスの CdS セルで検知することで実現しているが、この受光をもとに各楽器デバイス間で発音タイミングを同期させる Arduino プログラム (douki_saigo) の内部設計については、2.3.2 項（楽器デバイスの内部設計）で説明する。

2.3 楽器デバイス

2.3.1 概要

最後に、楽器デバイスについて説明する。楽器デバイスの構成図を図 18 に示す。本デバイスは、レーザー光を受光するための CdS セルと Arduino、そして Processing により楽曲を再生するための PC で構成され、1 つの楽器を再現するデバイスであり、主に以下の 3 つの機能を有する。本システムでは、ドラム、フルート、ストリングス、ピアノの 4 種類の楽器を再現するため、それぞれ 4 台ずつ用意した。

第一の機能は、楽器デバイス間の同期である。回転する中継機デバイスから照射されるレーザー光を CdS セルが受光する間隔から、各楽器デバイスは楽曲の BPM を推定する。そのうえで、ドラムを担当する楽器デバイスをマスター、ピアノ・ストリングス・フルートを担当する楽器デバイスをスレーブとした UDP 通信により、発音タイミングのずれを相互に送受信して補正することで、4 台の楽器デバイス間における同期を実現している。

第二の機能は、演奏の開始・終了および音量変更の受信である。これらの情報は、指揮デバイスからの UDP 通信によって各楽器デバイスへ直接送信される。

第三の機能は、楽曲の発音である。各楽器デバイスの Arduino に接続された PC 上で動作する Processing が、Arduino からの USB シリアル通信で受け取った発音指示（音程・音量等）をもとに波形を合成することで再生される。ここからは、内部設計について説明する。これらの同期通信および発音処理の詳細については 2.3.2 項で説明する。

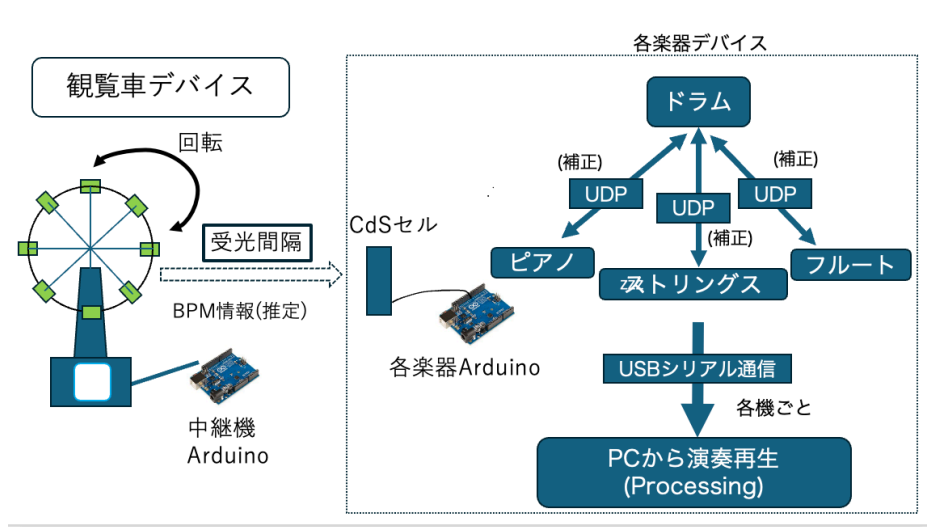


図 18 楽器デバイスのシステム構成図

2.3.2 内部設計

本節では、楽器デバイスにおける音色表現を担うソフトウェア（Processing）の内部設計について説明する。楽器デバイスは、Arduino から受信した発音指示をもとに、Processing 上で波形を合成して音を鳴らす構成となっている。合成方式は、あらかじめ実在の楽器音を FFT 解析して得た周波数・振幅データ（ルックアップテーブル、以下 LUT）をもとに多数の正弦波を重ね合わせる加算合成を基本とし、これに息や弦の摩擦音を模したノイズ成分を加えることで音色を再現している。

続いて、Processing プログラムのファイル構成を示す。表 20 の通り、各ファイルは機能ごとに分割されている。

表 20 Processing プログラムのファイル構成

ファイル名	概要
SoundProcessingCode.pde	setup/draw/serialEvent による全体制御, シリアル通信の受信処理, デバッグ用のキー入力処理
Utils.pde	LUT データの読み込みと保持, ピッチ・ベロシティに応じた倍音プロファイルの生成を行う DataUtils クラス
InstrumentPlayer.pde	発音命令を各楽器クラスへ委譲する InstrumentPlayer クラス
NoteSampler.pde	Arduino 未接続でも単体動作確認できる自動演奏機能, および内蔵の試奏用楽譜データ
AddEffect.pde	残響音 (リバーブ) エフェクトを付加する AddReverb クラス
Flute.pde	フルートの音色合成を行う FluteInstrument クラス
Strings.pde	ストリングス (弦楽器群) の音色合成を行う StringsInstrument クラス
Piano.pde	ピアノの音色合成を行う PianoInstrument クラス
Drum.pde	キック・スネア・ハイハットの音色合成を行う KickInstrument・SnareInstrument・HiHatInstrument クラス
Horn.pde	ホルンの音色合成を行う HornInstrument クラス
Trumpet.pde	トランペットの音色合成を行う TrumpetInstrument クラス

各楽器クラスは, Minim ライブラリの提供する Instrument インタフェースを実装しており, コンストラクタで音色を構築したうえで, noteOn 関数で発音を, noteOff 関数で消音を行う共通の枠組みに従っている. 発音命令は InstrumentPlayer クラスを介して各楽器固有のクラスへ委譲され, 波形生成に必要な LUT データは DataUtils クラスに集約されている. 最終的な音声信号は, 楽器の種類に応じて AddReverb による残響エフェクトを通したうえで AudioOutput へ出力される. プログラム全文は付録に示す.

ここからは, シリアル通信による Arduino との連携, LUT データの管理 (DataUtils), 発音命令の委譲 (InstrumentPlayer), 残響エフェクト (AddEffect), デバッグ用自動演奏 (NoteSampler), 各楽器クラスによる音色合成の順に, 使用する変数・関数の仕様と処理内容について説明する.

まず, Arduino とのシリアル通信および発音ディスパッチについて説明する. SoundProcessingCode.pde の setup 関数では, Minim ライブラリの初期化, 残響用バス (longOut・shortOut) の生成, InstrumentPlayer および DataUtils の初期化, そしてシリアルポートの初期化を行う. 楽器デバイスは 1 台の PC につき 1 種類の楽器を担当する構成となっており, 定数 arduinoNumber によって, この Processing インスタンスがどの楽器として動作するかを切り替える. Main (SoundProcessingCode.pde) で使用する主な変数の仕様を表 21 に示す.

表 21 Main (SoundProcessingCode.pde) で使用する主な変数の仕様一覧

変数名	型	説明
myPort	Serial	Arduino とのシリアル通信ポート
musicBPM	int	中継機から受信した現在の BPM 値を保持（発音長さの変換に使用）
currentMasterVolume	float	指揮デバイスから受信したマスターボリューム (0.2~1.0) を保持
arduinoNumber	final byte	この Processing インスタンスが担当する楽器を指定 (1: フルート, 2: ストリングス, 3: ピアノ系, 4: ドラム)
isDebugMode	boolean	キーボード操作によるデバッグ発音を許可するかのフラグ
longOut / shortOut	Summer	残響エフェクト用の合成バス。ストリングス・フルート等の持続音は longOut へ、ドラム等の打撃音は shortOut へパッチされる
player	InstrumentPlayer	発音命令を委譲する InstrumentPlayer のインスタンス
utils	DataUtils	LUT データを保持・提供する DataUtils のインスタンス

続いて、Main (SoundProcessingCode.pde) で使用する主な関数の仕様を表 22 に示す。

表 22 Main (SoundProcessingCode.pde) で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Minim の初期化、残響用バスの生成、InstrumentPlayer・DataUtils の初期化、シリアルポートの初期化を行う
draw	void	なし	オシロスコープおよび FFT スペクトルアナライザの描画を毎フレーム行う
serialEvent	void	Serial p	Arduino から改行区切りで届いたシリアルデータを解析し、ヘッダ ('N'/'B'/'V') に応じて発音・BPM 変更・音量変更の各処理を呼び出す
Play / keyPressed	void	なし	isDebugMode が true の場合にのみ有効な、キーボード操作による単体試奏・デバッグ用の関数

以下では、表 22 に示した関数のうち、実際の演奏に用いられる serialEvent 関数について、実際のコードとともに説明する。この関数は、Arduino から届いた 1 行のデータをカンマで分割し、先頭要素をヘッダとして判定する。ヘッダが'N'（発音指示）の場合、5 個 1 組（音程・長さ・開始ベロシティ・終了ベロシティ・種別）のデータを順に読み取り、duration には BPM に応じた時間換算 ($60000 \div (\text{BPM} \times 24)$) を施したうえで、arduinoNumber に応じた楽器の Play 関数を呼び出す。serialEvent 関数の発音指示部分をコード 29 に示す。

コード 29 serialEvent 関数（発音指示'N'の処理部分）

```

1  case "N" :
2      if (listLength % 5 != 0) break;
3      for (int i = 5; i <= listLength; i += 5) {
4          float pitch, duration, startVel, endVel, type;
5          try {
6              pitch    = float(list[i-4]);
7              duration = float(list[i-3]) * (2.5f / musicBPM); // ÷60000(BPM*24)
8              startVel = float(list[i-2]);
9              endVel   = float(list[i-1]);
10             type     = int(list[i]);
11         } catch (NumberFormatException e) {
12             continue;
13         }
14
15         switch (arduinoNumber) {
16             case 1 :
17                 player.PlayFlute(pitch, duration, startVel, endVel, currentMasterVolume);
18                 break;
19             case 2 :
20                 player.PlayStrings(pitch, duration, startVel, endVel, currentMasterVolume);
21                 break;
22             case 3 :
23                 if (type == 0) player.PlayPiano(pitch, duration, startVel,
24                     currentMasterVolume);
25                 else if (type == 1) player.PlayTrumpet(pitch, duration, startVel, endVel,
26                     currentMasterVolume);
27                 else if (type == 2) player.PlayHorn(pitch, duration, startVel, endVel,
28                     currentMasterVolume);
29                 break;
30             case 4 :
31                 int a = int(list[i-4]) % 12;
32                 if (a == 0) player.PlayKick(currentMasterVolume, startVel);
33                 else if (a == 3) player.PlaySnare(currentMasterVolume, startVel, endVel);
34                 else if (a == 4) player.PlayHiHat(currentMasterVolume, startVel);
35                 break;
36             default: break;
37         }
38     }
39     break;

```


ここで、種別 type による分岐は、ハッカソン第 11 回時点でトランペット (TrumpetInstrument) とホルン (HornInstrument) を追加したことに伴い、楽譜データ上で発音する楽器の種類を判別するために新設されたものである。また、ヘッダが'B' (BPM 変更) の場合は musicBPM を 60~180 の範囲に制限して更新し、ヘッダが'V' (マスターボリューム変更) の場合は currentMasterVolume を 0.2~1.0 の範囲に写像して更新する。

次に、楽器デバイスにおける LUT データの管理 (DataUtils クラス) について説明する。本デバイスでは、アイオワ大学等が公開する実楽器の音声データを FFT 解析し、各音程・各強弱 (pp/mf/ff) ごとの「周波数と振幅のペア」を JSON 形式の LUT として事前に用意している。DataUtils クラスは、この LUT の読み込みと、発音時のピッチ・ベロシティに応じた倍音プロファイルの動的な取得を担う。DataUtils クラスで使用する主な変数の仕様を表 23 に示す。

表 23 DataUtils クラスで使用する主な変数の仕様一覧

変数名	型	説明
note	float[][]	NoteSampler が参照する試奏用楽譜データ (2 次元 LUT)
fluteFFLUT / MFLUT / PPLUT 等	float[][][]	各楽器・各強弱ごとの 3 次元 LUT (音程インデックス × ピーク数 × [周波数, 振幅])。フルート・バイオリン・ビオラ・チェロ・コントラバス・ピアノ・ホルン・トランペットの各強弱分を保持
pianoDecayLUT 等	float[][]	ピアノ専用の減衰時間・うなり速度・うなり深さの 2 次元 LUT (強弱ごとに 3 種類 × 3 強弱の計 9 個)
kickLUT / snareLUT / hihat-LUT	float[][]	ドラム各パートの [周波数, 強度, 減衰時間] を格納する 2 次元 LUT

続いて、DataUtils クラスで使用する主な関数の仕様を表 24 に示す。

表 24 DataUtils クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
Load2LUT	float[][]	String fileName	JSON ファイルを読み込み, float[][] の 2 次元 LUT へ変換する
Load3LUT	float[][][]	String fileName	JSON ファイルを読み込み, float[][][] の 3 次元 LUT ([周波数, 振幅] のペア配列) へ変換する
GetProfileForPitch	float[][]	float pitch, float[][][] lut, int baseMidiNote	指定ピッチに最も近い実測データを LUT から探索し, 元データとの半音差に応じて周波数を比例スケリング (ピッチスケリング) したうえで返す
GetDynamicProfile	float[][]	float pitch, float velocity, String type, int baseMidiNote, float basePower	pp/mf/ff の 3 つのプロファイルをベロシティに応じて選択・合成し, 振幅 0.05 未満のピークを除去したうえで, 目標音量に基づくゲイン正規化を行った最終的な [周波数, 音量] 配列を返す
GetLUT	float[][]	String type	文字列で指定した種類の LUT (ピアノの減衰特性やドラムの LUT など) を返す統合ゲッター

以下では, 表 24 に示した関数のうち, 本デバイスの音色再現の核となる GetProfileForPitch 関数と GetDynamicProfile 関数について, 実際のコードとともに説明する. はじめに, GetProfileForPitch

関数について説明する。この関数は、要求されたピッチに最も近い有効なサンプルインデックスを LUT から探索し、そのサンプルが本来持つ基準音程 (sourceMidi) と要求ピッチとの半音差から求めた倍率 (pitchShiftFactor) を、実測された周波数構造にそのまま乗算することで、インハーモニシティ (倍音の非整数性) のデコボコを保持したまま滑らかにピッチをスケーリングする。GetProfileForPitch 関数の要部をコード 30 に示す。

コード 30 DataUtils::GetProfileForPitch 関数 (要部)

```

1 float[][] GetProfileForPitch(float pitch, float[][] lut, int baseMidiNote) {
2     float originalPitch = pitch;
3     float maxMidiNote = baseMidiNote + lut.length - 1;
4     float searchPitch = constrain(pitch, baseMidiNote, maxMidiNote);
5
6     int targetIndex = round(searchPitch) - baseMidiNote;
7     targetIndex = constrain(targetIndex, 0, lut.length - 1);
8
9     // targetIndexがnull (データ欠損) の場合は、最も近い有効なインデックスを探索する
10    int finalIndex = targetIndex;
11    // (探索処理は省略。詳細は付録のプログラム全文を参照)
12
13    float[][] sourceProfile = lut[finalIndex];
14    int numPeaks = sourceProfile.length;
15    float[][] result = new float[numPeaks][2];
16
17    // 流元サンプルの基準MIDIノート番号を逆算
18    float sourceMidi = finalIndex + baseMidiNote;
19    // 要求ピッチと流元との半音差から、周波数の倍率を算出
20    float pitchShiftFactor = pow(2.0f, (originalPitch - sourceMidi) / 12.0f);
21
22    for (int h = 0; h < numPeaks; h++) {
23        result[h][0] = sourceProfile[h][0] * pitchShiftFactor; // 実測周波数構造をそのま
24                                                                // まスケーリング
25        result[h][1] = sourceProfile[h][1]; // 振幅はそのままコピー
26    }
27    return result;
28 }
```

次に、GetDynamicProfile 関数について説明する。この関数は、まず pp/mf/ff の 3 つの強弱プロファイルを GetProfileForPitch 関数で取得したうえで、ベロシティを 0~1 に正規化し、あらかじめ定めた 3 つの閾値 (32/80/112 を 127 で除した値) と比較して、隣接する 2 つの強弱のうちベロシティに近い方のプロファイル形状を採用する。続いて、振幅 0.05 未満の微小なピークを除去したうえで、採用した各ピークの振幅の 2 乗和 (エネルギー) に対する目標音量の比からゲインを算出し、全ピークへ一括で乗算することで最終的な音量を決定する。GetDynamicProfile 関数の要部をコード 31 に示す。

コード 31 DataUtils::GetDynamicProfile 関数 (要部)

```

1 float[][] GetDynamicProfile(float pitch, float velocity, String type, int
```

```

    baseMidiNote, float basePower) {
2  float[][] profilePP, profileMF, profileFF;
3  // switch(type)によりPP/MF/FFの各LUTからGetProfileForPitchで取得（詳細略）
4
5  float normVel = constrain(velocity, 0, 127) / 127.0f;
6  float thresholdPP = 32.0f / 127.0f;
7  float thresholdMF = 80.0f / 127.0f;
8  float thresholdFF = 112.0f / 127.0f;
9
10 float[][] baseProfile;
11 float t = 0;
12 if (normVel <= thresholdPP) {
13     baseProfile = profilePP;
14 } else if (normVel <= thresholdMF) {
15     t = map(normVel, thresholdPP, thresholdMF, 0.0f, 1.0f);
16     baseProfile = (t < 0.5f) ? profilePP : profileMF;
17 } else if (normVel <= thresholdFF) {
18     t = map(normVel, thresholdMF, thresholdFF, 0.0f, 1.0f);
19     baseProfile = (t < 0.5f) ? profileMF : profileFF;
20 } else {
21     baseProfile = profileFF;
22 }
23
24 int numPeaks = baseProfile.length;
25 float ampThreshold = 0.05f;
26 int validPeakCount = 0;
27 float energy = 0;
28 float[][] finalProfile = new float[numPeaks][2]; // 実際は有効数分で確保
29
30 for (int h = 0; h < numPeaks; h++) {
31     if (baseProfile[h][1] < ampThreshold) continue; // 微小ピークは除去
32     finalProfile[validPeakCount][0] = baseProfile[h][0];
33     finalProfile[validPeakCount][1] = baseProfile[h][1];
34     energy += baseProfile[h][1] * baseProfile[h][1];
35     validPeakCount++;
36 }
37
38 float targetVolume = basePower * pow(normVel, 0.9f);
39 float gain = targetVolume / sqrt(energy + 1e-9f);
40 for (int i = 0; i < validPeakCount; i++) {
41     finalProfile[i][1] *= gain; // エネルギー正規化ゲインを適用
42 }
43 return finalProfile;
44 }

```

次に、発音命令の委譲を行う InstrumentPlayer クラスについて説明する。このクラスは、AudioOutput への参照を保持し、各 PlayXxx 関数が呼び出されるたびに、対応する楽器クラスの

インスタンスをその場で生成して out.playNote 関数へ渡すという単純な委譲処理のみを行う。InstrumentPlayer クラスで使用する主な関数の仕様を表 25 に示す。

表 25 InstrumentPlayer クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
PlayFlute PlayStrings / Play- Horn / PlayTrumpet	/ void	float pitch, float duration, float startVel, float endVel, float masterVol	音程・長さ・開始ベロシティ・ 終了ベロシティ・マスターボ リュームを引数に取り、対応 する Instrument のインスタ ンスを out.playNote へ渡す
PlayPiano	void	float pitch, float velocity, float masterVol	音程・ベロシティ・マスター ボリュームを引数に取り、 PianoInstrument を再生する (開始・終了ベロシティの区 別はない)
PlayKick / PlaySnare / PlayHiHat	void	float masterVol, float veloc- ity 等	マスターボリューム・ベ ロシティ等を引数に取り、 対応するドラムパートの Instrument を再生する

InstrumentPlayer クラスの PlayFlute 関数をコード 32 に示す。他の PlayXxx 関数も、生成する Instrument クラスが異なるのみで、同様の構造となっている。

コード 32 InstrumentPlayer::PlayFlute 関数

```

1 void PlayFlute(float midiNote, float duration, float startVel, float endVel, float
  masterVol) {
2   out.playNote(0.0f, duration,
3     new FluteInstrument(midiNote, duration, startVel, endVel, masterVol));
4 }

```

次に、残響エフェクトを付加する AddEffect.pde の AddReverb クラスについて説明する。このクラスは Minim ライブラリの UGen を継承しており、講義で扱われたたたみ込み積分（コンボリューション）を、指数関数的に減衰するランダムノイズを疑似的なインパルス応答として用いることで実装している。左右のチャンネルの記憶バッファ（historyBufferL・historyBufferR）を独立させることで、ステレオ音場に対応している。AddReverb クラスの心臓部である uGenerate 関数をコード 33 に示す。

コード 33 AddReverb::uGenerate 関数

```

1 protected void uGenerate(float[] channels) {
2   float[] inputs = audio.getLastValues();
3   float inL = inputs[0];
4   float inR = (inputs.length > 1) ? inputs[1] : inL;

```

```

5
6 historyBufferL[bufferIndex] = inL;
7 historyBufferR[bufferIndex] = inR;
8
9 float outL = 0;
10 float outR = 0;
11 for (int j = 0; j < kernelSize; j++) {
12     int idx = (bufferIndex - j + kernelSize) % kernelSize;
13     outL += historyBufferL[idx] * impulseResponse[j];
14     outR += historyBufferR[idx] * impulseResponse[j];
15 }
16
17 bufferIndex = (bufferIndex + 1) % kernelSize;
18
19 if (channels.length == 1) {
20     channels[0] = inL + outL;
21 } else if (channels.length >= 2) {
22     channels[0] = inL + outL;
23     channels[1] = inR + outR;
24 }
25 }

```

setup 関数内では、持続音向けの残響バス longOut（カーネルサイズ 15000，レベル 0.20）と、打撃音向けの残響バス shortOut（カーネルサイズ 2000，レベル 0.25）の 2 種類の AddReverb が生成されており、各楽器クラスは noteOn 関数内で masterEnv をこのいずれかへパッチすることで、音色に適した残響を選択している。

次に、デバッグ用の自動演奏機能を提供する NoteSampler.pde について説明する。本機能は、指揮デバイスや中継機デバイスが未接続の状態でも PC 単体で音色の検証を行えるようにするためのものであり、Processing の標準描画ループ（draw）に依存すると波形描画の負荷でタイミングがずれるため、音楽専用のスレッド（audioThread）を用いて高精度にタイミングを管理している。NoteSampler.pde で使用する主な変数の仕様を表 26 に示す。

表 26 NoteSampler.pde で使用する主な変数の仕様一覧

変数名	型	説明
tick	volatile int	現在の演奏位置（24 分音符単位の tick 数）
interval	volatile long	1 tick あたりのナノ秒単位の時間間隔
noteIndex	volatile int	Note 配列内で次に処理する音符のインデックス
returnTick[][]	volatile int[][]	ループ再生のためのジャンプ情報（トリガー tick, ジャンプ先 tick, ジャンプ先 noteIndex）を格納する配列
isPlaying	volatile boolean	自動演奏中かどうかを示すフラグ
audioThread	Thread	高精度なタイミング管理を行う音楽専用スレッド
Note[][]	int[][]	試奏用に内蔵された楽譜データ（tick, 長さ, 音程, 開始ベロシティ, 終了ベロシティ, 種別）

続いて、NoteSampler.pde で使用する主な関数の仕様を表 27 に示す。

表 27 NoteSampler.pde で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
GetInterval	long	int playBPM	指定 BPM から 1 tick あたりのナノ秒間隔を算出する ($60000000000 \div (\text{BPM} \times 24)$)
SoundPlay	void	なし	音楽専用スレッドから毎 tick 呼び出され、returnTick によるループ判定を行った後、現在の tick に該当する音符を Note 配列から取得して timbre に応じた PlayXxx 関数を呼び出す

SoundPlay 関数をコード 34 に示す。

コード 34 SoundPlay 関数

```

1 void SoundPlay() {
2   if (tick == returnTick[returnIndex][0]) {
3     tick = returnTick[returnIndex][1];
4     noteIndex = returnTick[returnIndex][2];
5     returnIndex++;
6     if (returnIndex == returnTick.length) returnIndex = 0;
7   }
8 }

```

```

9  if (noteIndex < Note.length) {
10     float durationTuning = interval / 1000000000.0f;
11     while (Note[noteIndex][0] == tick) {
12         float duration = float(Note[noteIndex][1]) * durationTuning;
13         float pitch = float(Note[noteIndex][2]);
14         float startVelocity = float(Note[noteIndex][3]);
15         float endVelocity = float(Note[noteIndex][4]);
16         float type = (Note[noteIndex].length == 6) ? float(Note[noteIndex][5]) : 0;
17         switch (timbre) {
18             case "Flute": case "Flute2":
19                 player.PlayFlute(pitch, duration, startVelocity, endVelocity, 1.0f);
20                 break;
21             // 以下、Strings/Piano/Drum/Horn/Trumpetも同様に分岐（詳細は付録参照）
22         }
23         noteIndex ++;
24         if (noteIndex == Note.length) break;
25     }
26 }
27 }

```

最後に、各楽器クラスによる音色合成について説明する。いずれの楽器クラスも、コンストラクタ内で DataUtils から取得した倍音プロファイルをもとに正弦波（Oscil）を加算合成し、ADSR（Attack, Decay, Sustain, Release）エンベロープおよび Line による時間変化を付加したうえでミキサー（Summer）へ結線し、noteOn 関数でエンベロープと Line を一斉に起動する、という共通の設計に従っている。以下では、各楽器クラスに特有の合成技法について、実際のコードとともに説明する。

フルート（FluteInstrument） まず、フルートの音色合成について説明する。FluteInstrument クラスは、DataUtils から取得した倍音プロファイルをもとに正弦波を加算合成する層と、息が管に吹き込まれる音（Chiff）を再現するノイズ層の 2 層構造で構成される。加算合成の各倍音には、globalBreathSpeed という 1 つの共通の乱数速度で駆動されるビブラート（FM 変調）と、倍音ごとに個別の深さを持つ音量の呼吸的揺らぎ（AM 変調）が付加されている。FluteInstrument クラスで使用する主な変数の仕様を表 28 に示す。

表 28 FluteInstrument クラスで使用する主な変数の仕様一覧

変数名	型	説明
numHarmonics	byte	実測データから動的に決定される倍音 (Oscil) の数
masterEnv	ADSR	音全体の発音・消音を制御するエンベロープ
harmonicEnvs[]	ADSR[]	各倍音の立ち上がりを個別に制御するエンベロープ配列
ampLines[]	Line[]	開始ベロシティと終了ベロシティの間で各倍音の音量を滑らかに変化させる Line 配列
wobbleLines[] / wobble-Factors[]	Line[] / float[]	倍音ごとの音量揺らぎ (AM 変調) の深さをフェードさせる Line 配列と、その揺らぎ比率
breathEnv	ADSR	息のノイズ (Chiff) 成分のエンベロープ

続いて、FluteInstrument クラスで使用する主な関数の仕様を表 29 に示す。

表 29 FluteInstrument クラスで使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
FluteInstrument (コンストラクタ)	なし	float midiNote, float duration, float startVel, float endVel, float masterVol	DataUtils から取得した倍音プロファイルをもとに、加算合成層と息ノイズ層を構築する
noteOn	void	float dur	masterEnv・harmonicEnvs を起動し、ampLines・wobbleLines を開始値から終了値へフェードさせ、masterEnv を longOut へパッチする
noteOff	void	なし	masterEnv および harmonicEnvs を消音し、リリース完了後に longOut からアンパッチする

以下では、コンストラクタと noteOn 関数について、実際のコードとともに説明する。まず、加算合成部分について説明する。各倍音について Oscil を生成し、位相をランダムに散らしたうえで、ビブラート用の Oscil (freqController) を周波数へ、音量揺らぎ用の Oscil (ampLFO) を振幅へそれぞれパッチしている。加算合成部分をコード 35 に示す。

コード 35 FluteInstrument コンストラクタ (加算合成部分)

```
1 for (int i = 0; i < numHarmonics; i++) {
2     float targetFreq = startProfile[i][0];
3     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
4     osc.setPhase(random(1.0f)); // アタック音の機械的な均一さを防ぐ
5
6     float baseAmp = startProfile[i][1] * volFactor;
7     startAmps[i] = baseAmp * startGainFactor;
8     endAmps[i] = baseAmp * endGainFactor;
9
10    // ビブラート (FM変調)
11    float vibDepthHz = targetFreq * 0.0015f;
12    Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz/10, Waves.SINE);
13    freqController.offset.setLastValue(targetFreq);
14    freqController.patch(osc.frequency);
15
16    // 音量揺らぎ (AM変調) : ウナリ自体をフェードさせる
17    wobbleFactors[i] = wobbleDepths[i];
18    wobbleLines[i] = new Line();
19    Oscil ampLFO = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE);
20    ampLFO.setPhase(random(1.0f));
21    wobbleLines[i].patch(ampLFO.amplitude);
22    ampLFO.patch(osc.amplitude);
23
24    ampLines[i] = new Line();
25    ampLines[i].patch(osc.amplitude);
26
27    float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
28    harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
29    osc.patch(harmonicEnvs[i]);
30    harmonicEnvs[i].patch(mixer);
31 }
```

次に、息のノイズ (Chiff) 部分について説明する。ホワイトノイズをバンドパスフィルター (MoogFilter) に通すことで、息が管に吹き込まれる瞬間の「シュッ」という音のみを抽出している。息ノイズ部分をコード 36 に示す。

コード 36 FluteInstrument コンストラクタ (息ノイズ部分)

```
1 float startNorm = constrain(startVel, 0, 127) / 127.0f;
2 float attackNoiseVol = masterVol * 0.008f * pow(startNorm, 0.9f);
3 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
4
5 MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);
```

```

6 breathFilter.type = MoogFilter.Type.BP;
7
8 breathEnv = new ADSR(1.0f, 0.001f, 0.2f, 0.0f, 0.0f);
9 breathNoise.patch(breathFilter);
10 breathFilter.patch(breathEnv);
11 breathEnv.patch(mixer);

```

最後に、noteOn 関数について説明する。この関数は、masterEnv と harmonicEnvs を起動したうえで、ampLines（ベース音量）と wobbleLines（揺らぎの深さ）を、音符の長さに 0.5 秒の余裕を加えた safeDur にわたって開始値から終了値へフェードさせる。noteOn 関数をコード 37 に示す。

コード 37 FluteInstrument::noteOn 関数

```

1 void noteOn(float dur) {
2     masterEnv.noteOn();
3     float safeDur = dur + 0.5f;
4
5     for(int i = 0; i < numHarmonics; i++) {
6         harmonicEnvs[i].noteOn();
7         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
8         if (wobbleLines[i] != null) {
9             float factor = wobbleFactors[i];
10            wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
11        }
12    }
13    breathEnv.noteOn();
14    masterEnv.patch(longOut);
15 }

```

ストリングス (StringsInstrument) 次に、ストリングスの音色合成について説明する。StringsInstrument クラスは、単一の楽器データではなく、コントラバス・チェロ・ビオラ・バイオリンの 4 種類の LUT を音程に応じてクロスフェード（混合）させることで、低音から高音まで滑らかに音色が変化する弦楽器群を表現している点が最大の特徴である。音程が低いほどコントラバスの比率が高く、音程が上がるにつれてチェロ・ビオラ・バイオリンへと比率が滑らかに移行する。StringsInstrument クラスで使用する主な変数・関数の仕様を表 30 に示す。

表 30 StringsInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
maxTotalOscils (変数)	4 楽器分の混合に対応するため多めに確保した Oscil の最大本数 (40)
totalActiveOscils (変数)	実際に生成された Oscil の本数
createInstrumentOscillators (関数)	1 つの楽器 (コントラバス等) の LUT プロファイルを、振幅の大きい順に上限本数まで選別したうえで Oscil 群として生成し、ミキサーへ接続する
noteOn / noteOff (関数)	生成された totalActiveOscils 本の Oscil に対して、音量・ビブラート・音量揺らぎのフェードを一括で起動・終了する

まず、楽器ブレンド比率の算出について説明する。音程 (midiNote) の値に応じて、隣接する 2 楽器間の比率を map 関数で線形補間する。ブレンド比率の算出部分をコード 38 に示す。

コード 38 StringsInstrument コンストラクタ (楽器ブレンド比率の算出)

```

1  float baseBassRatio = 0.0f, baseCelloRatio = 0.0f, baseViolaRatio = 0.0f,
    baseViolinRatio = 0.0f;
2
3  if (midiNote <= 36) {
4      baseBassRatio = 1.0f;
5  } else if (midiNote <= 48) {
6      baseBassRatio = map(midiNote, 36, 48, 1.0f, 0.0f);
7      baseCelloRatio = 1.0f - baseBassRatio;
8  } else if (midiNote <= 60) {
9      baseCelloRatio = map(midiNote, 48, 60, 1.0f, 0.0f);
10     baseViolaRatio = 1.0f - baseCelloRatio;
11 } else if (midiNote <= 72) {
12     baseViolaRatio = map(midiNote, 60, 72, 1.0f, 0.0f);
13     baseViolinRatio = 1.0f - baseViolaRatio;
14 } else {
15     baseViolinRatio = 1.0f;
16 }

```

次に、createInstrumentOscillators 関数について説明する。この関数は、指定された楽器の LUT プロファイルを振幅の大きい順にソートしたうえで上限本数 (maxPeaks) まで選別し、周波数の低い順に並べ直してから Oscil として生成する。これにより、計算負荷を抑えつつ音色に対する寄与の大きい成分を優先的に採用している。createInstrumentOscillators 関数の要部をコード 39 に示す。

コード 39 StringsInstrument::createInstrumentOscillators 関数 (要部)

```

1  private void createInstrumentOscillators(final float[][] profile, float blendRatio,
    int maxPeaks,
2      float volFactor, float globalVibSpeed, float fmDepthCents, Summer mixer,
3      float startGainFactor, float endGainFactor) {
4      if (profile == null || profile.length == 0) return;

```

```

5
6 // 振幅の大きい順にインデックスをソートし、上位limitPeaks個を抽出（詳細は付録参照）
7 // その後、周波数の低い順に並べ直す
8
9 for (int h = 0; h < limitPeaks; h++) {
10     if (totalActiveOscils >= maxTotalOscils) break;
11
12     float targetFreq = filteredProfile[h][0];
13     float rawAmp      = filteredProfile[h][1];
14     if (targetFreq > 14000.0f) continue;
15
16     // 7000Hz~17000Hzにかけて振幅を滑らかに減衰させる高域フィルター
17     if (targetFreq > 7000.0f) {
18         float filterFactor = map(targetFreq, 7000.0f, 17000.0f, 1.0f, 0.0f);
19         rawAmp *= constrain(filterFactor, 0.0f, 1.0f);
20     }
21
22     float baseAmp = rawAmp * volFactor * blendRatio;
23     int idx = totalActiveOscils;
24     startAmps[idx] = baseAmp * startGainFactor;
25     endAmps[idx]   = baseAmp * endGainFactor;
26
27     oscils[idx] = new Oscil(targetFreq, 0.0f, Waves.SINE);
28     oscils[idx].setPhase(random(1.0f));
29     oscils[idx].patch(mixer);
30     totalActiveOscils++;
31 }
32 }

```

ピアノ (PianoInstrument) 次に、ピアノの音色合成について説明する。PianoInstrument クラスは、1つの音に対して2本の弦をわずかにデチューン（周波数をずらす）して重ねる「デュアル・ストリング構造」により、時間が経つにつれて自然に生じる音のうなり（Beat）を再現している点が特徴である。また、実測データ（LUT）が存在しない倍音・音域については、ピアノの物理特性の傾向を数式化した近似モデルで補完するハイブリッド方式を採用している。PianoInstrument クラスで使用する主な変数・関数の仕様を表 31 に示す。

表 31 PianoInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
masterEnv (変数)	ダンパー（離鍵時のミュート）の挙動を表す全体エンベロープ
hammerEnv (変数)	ハンマーが弦を叩く瞬間のノイズ用エンベロープ
damps (変数)	芯の弦・共鳴弦それぞれの Damp（減衰）オブジェクトを保持する ArrayList
PianoInstrument (コンストラクタ)	ベロシティ層の決定、倍音ごとのデュアル・ストリング構築、ハンマーノイズの生成を行う
noteOn / noteOff (関数)	masterEnv・hammerEnv および全 damps 要素を一斉に起動・終了する

以下では、デュアル・ストリング構造の構築部分について説明する。各倍音について、アタックの芯となるレイヤー A (coreOsc) と、時間差で立ち上がる共鳴弦のレイヤー B (resOsc) の 2 層を生成する。レイヤー B はレイヤー A よりわずかに周波数をずらし (wobbleRate 分のデチューン)、buildUpTime 秒かけてゆっくり音量を上げることで、芯の音と干渉して自然なうなりを生む。デュアル・ストリング構造の構築部分をコード 40 に示す。

コード 40 PianoInstrument コンストラクタ (デュアル・ストリング構造)

```

1 // レイヤーA：アタックの芯（Dry弦）
2 Oscil coreOsc = new Oscil(targetFreq, amp, Waves.SINE);
3 coreOsc.setPhase(random(0.0f, 0.1f));
4 Damp coreDamp = new Damp(0.001f, decayTime);
5 coreOsc.patch(coreDamp).patch(mixer);
6 damps.add(coreDamp);
7
8 // レイヤーB：時間差で来る響き（共鳴弦）
9 float reverbAmp = amp * wobbleDepth;
10 if (reverbAmp > 0.0001f && wobbleRate > 0.0f) {
11     // 周波数をわずかにズラす（Detune）ことで、芯の弦と干渉して自然な「うなり(Beat)」を生む
12     Oscil resOsc = new Oscil(targetFreq + wobbleRate, reverbAmp, Waves.SINE);
13     Damp resDamp = new Damp(buildUpTime, decayTime);
14     resOsc.patch(resDamp).patch(mixer);
15     damps.add(resDamp);
16 }

```

続いて、ハンマーの打撃ノイズについて説明する。音程の高低に応じてフィルターのカットオフを変化させることで、高音の鍵盤ほど硬い「カチッ」としたノイズになるよう調整している。ハンマーノイズ部分をコード 41 に示す。

コード 41 PianoInstrument コンストラクタ (ハンマーノイズ部分)

```

1 float hammerVol = masterVol * 0.003f * normVel;
2 Noise hammerNoise = new Noise(hammerVol, Noise.Tint.WHITE);

```

```

3
4 float hammerCutoff = map(midiNote, 21, 108, 600f, 4000f);
5 MoogFilter hammerFilter = new MoogFilter(hammerCutoff, 0.1f);
6 hammerFilter.type = MoogFilter.Type.LP;
7
8 hammerEnv = new ADSR(1.0f, 0.001f, 0.02f, 0.0f, 0.01f);
9 hammerNoise.patch(hammerFilter).patch(hammerEnv);

```

ドラム (KickInstrument・SnareInstrument・HiHatInstrument) 次に、ドラムの音色合成について説明する。ドラムはフルート等の旋律楽器とは異なり、Instrument クラスをキック・スネア・ハイハットの3種類（および過去に試作された SnareInstrumentOld・HiHatInstrumentOld の旧版、未使用の CymbalInstrument）に分けて実装している。いずれも短時間で減衰する打撃音であるため、全体の出力は shortOut バスへパッチされる。3種の主要な変数・関数の仕様を表32に示す。

表 32 ドラム関連クラスで使用する主な変数・関数の仕様一覧

名称	説明
KickInstrument (コンストラクタ)	kickLUT の各成分について、衝撃の瞬間に高い周波数から本来の周波数へ急降下するピッチスweep (Line) を適用した Oscil を最大 50 本生成する
SnareInstrument (コンストラクタ)	胴鳴り (Body)、倍音 (Overtone)、スナッピー (ノイズ) の3パートを個別の Oscil / Noise とフィルターで構築する
HiHatInstrument (コンストラクタ)	実測ピークに基づく複数の金属共鳴用 Oscil と、高域ノイズによる打撃摩擦音を合成する
noteOn / noteOff (各クラス共通)	各パートのエンベロープを一斉に起動・終了し、masterEnv を shortOut へパッチ／アンパッチする

以下では、キックのピッチスweepと、スネアの胴鳴りパートについて、実際のコードとともに説明する。まず、KickInstrument のピッチスweepについて説明する。各周波数成分について、Line を用いて 0.01 秒かけて「本来の周波数の 1.5 倍」から「本来の周波数」へ直線的に下げること

で、太鼓の膜が叩かれた瞬間にピッチが下がっていく物理的な挙動を再現している。該当部分をコード 42 に示す。

コード 42 KickInstrument コンストラクタ (ピッチスweep部分)

```

1 for (int i = 0; i < numComponents; i++) {
2   if (kickLUT[i] == null || kickLUT[i].length < 3) continue;
3
4   float freq = kickLUT[i][0] * random(0.95f, 1.05f);
5   float mag = kickLUT[i][1];
6   float decayTime = kickLUT[i][2];
7   float amp = (baseAmp * mag) / numComponents;
8
9   tones[i] = new Oscil(freq, amp, Waves.SINE);
10  harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime*0.5f, 0.0f, 0.1f);

```

```

11
12 // ピッチスweep: 1.5倍の高さから本来の周波数へ急降下させる
13 pitchSweeps[i] = new Line(0.01f, freq * 1.5f, freq);
14 pitchSweeps[i].patch(tones[i].frequency);
15
16 tones[i].patch(harmonicEnvs[i]);
17 harmonicEnvs[i].patch(mixer);
18 }

```

次に、SnareInstrument の胴鳴りパートについて説明する。発音の瞬間に約 3 倍の周波数から本来の胴鳴り周波数 (172.3 Hz) へ急降下させるピッチエンベロープにより、スネアドラム特有のアタック感を表現している。該当部分をコード 43 に示す。

コード 43 SnareInstrument コンストラクタ (胴鳴りパート)

```

1 float bodyFreq      = 172.3f * random(0.99f, 1.01f); // 実測データの最強ピーク
2 float bodyPitchMod  = 3.0f; // アタック時は約516Hzから172.3Hzへ急降下
3 float bodySweepTime = 0.025f;
4 float bodyDecay      = 0.07f;
5 float bodyVolRatio   = 0.63f * sinSet;
6
7 bodyOsc = new Oscil(bodyFreq, baseAmp * bodyVolRatio, Waves.SINE);
8 bodyPitchEnv = new Line(bodySweepTime, bodyFreq * bodyPitchMod, bodyFreq);
9 bodyPitchEnv.patch(bodyOsc.frequency);
10 bodyAmpEnv = new ADSR(1.0f, 0.001f, bodyDecay, 0.0f, 0.05f);
11 bodyOsc.patch(bodyAmpEnv).patch(mixer);

```

ホルンとトランペット (HornInstrument・TrumpetInstrument) 最後に、ホルンおよびトランペットの音色合成について説明する。両クラスは、ハッカソン第 11 回時点で追加された金管楽器であり、事前課題の報告の通り、既存の FluteInstrument クラスの加算合成・ビブラート・息ノイズの構造をそのまま流用したうえで、KickInstrument で用いていたアタック時のピッチエンベロープ機能を新たに付加して作成されている。HornInstrument と TrumpetInstrument は、基準 MIDI ノートや各種パラメータの数値が異なるのみで、クラス構造自体は同一である。両クラスで使用する主な変数・関数の仕様を表 33 に示す。

表 33 HornInstrument・TrumpetInstrument クラスで使用する主な変数・関数の仕様一覧

名称	説明
pitchEnvLines[] (変数)	アタック時のピッチ急降下を制御する Line 配列 (Flute 由来の構造に新規追加)
targetFreqs[] (変数)	各倍音の本来の周波数を保持する配列, noteOn 時のピッチエンベロープ計算に使用
HornInstrument / TrumpetInstrument (コンストラクタ)	Flute 同様の加算合成・ビブラート・息ノイズ構造に加え, 各倍音の pitchEnvLines を Oscil の周波数 LFO の offset へパッチする
noteOn (関数)	masterEnv・harmonicEnvs の起動と ampLines・wobbleLines のフェードに加え, pitchEnvLines を「本来周波数の (1+pitchMod) 倍」から「本来周波数」へ sweepTime 秒でフェードさせる

以下では、両クラスに共通するアタック時のピッチエンベロープ機能について、HornInstrument の noteOn 関数を例に説明する。発音直後、各倍音の周波数を本来値より一時的に高く（ホルンでは pitchMod=0.05、すなわち 5%増しから）設定し、sweepTime 秒（ホルンでは 0.12 秒）かけて本来の周波数へ直線的に戻すことで、金管楽器特有の「音の頭が一瞬上ずってから定まる」挙動を再現している。該当部分をコード 44 に示す。なお、トランペットでは同じ構造に対して pitchMod=0.03、sweepTime=0.07 秒という異なる数値が用いられている。

コード 44 HornInstrument::noteOn 関数 (ピッチエンベロープ部分)

```

1 float sweepTime = 0.12f; // 変化にかかる時間
2 float pitchMod = 0.05f; // 本来の周波数に対する上乗せ割合
3
4 for(int i = 0; i < numHarmonics; i++) {
5     harmonicEnvs[i].noteOn();
6     ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
7     if (wobbleLines[i] != null) {
8         float factor = wobbleFactors[i];
9         wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
10    }
11
12    // アタック時のピッチエンベロープ (周波数の急降下)
13    if (pitchEnvLines[i] != null) {
14        float baseFreq = targetFreqs[i];
15        float startFreq = baseFreq * (1.0f + pitchMod); // 本来の周波数にpitchMod分を上乗
16        // せした値からスタート
17        pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq); // baseFreqへ一直線に
18        // 下げる
19    }
20 }

```

プログラム全文は付録に示す。

楽器デバイスの同期方式 (douki_saigo) 次に、各楽器デバイスの同期方式を担う Arduino プログラムについて説明する。楽器デバイスは、指揮デバイスからの演奏開始指示を受けた後、中継機デバイスの回転に伴うレーザー光を CdS セルで検知することでおおまかな BPM を推定し、さらに楽器機同士の UDP 通信によって発音タイミングを細かく補正することで、複数台の Arduino 間での高精度な同期を実現している。この同期方式では、ドラムを担当する Arduino を基準時刻源 (マスター) とし、ピアノ・ストリングス・フルートを担当する 3 台の Arduino をスレーブとするマスタースレーブ方式が採用されている。

続いて、同期プログラムのファイル構成を示す。表 34 の通り、各ファイルは担当する楽器ごとに用意されている。

表 34 同期プログラム (douki_saigo) のファイル構成

ファイル名	概要
drum_0706.ino	ドラム担当機のプログラム。同期のマスターとして動作し、全楽器機の発音タイミングを管理する
flute_0706.ino	フルート担当機のプログラム。同期のスレーブとして動作する
piano_0706.ino	ピアノ担当機のプログラム。同期のスレーブとして動作する
strings_0706.ino	ストリングス担当機のプログラム。同期のスレーブとして動作する

flute_0706.ino・piano_0706.ino・strings_0706.ino の 3 ファイルは、IP アドレス・楽器 ID・CdS 検知しきい値・埋め込みの楽譜データが異なるのみで、通信・同期ロジックの構造は完全に同一である。そのため、以下ではスレーブ側の代表例として flute_0706.ino を用いて説明し、マスター側は drum_0706.ino を用いて説明する。

次に、各 Arduino 間の同期に用いられる通信ヘッダの一覧を示す。表 35 の通り、指揮デバイスからの一斉指示に加え、マスター・スレーブ間で複数種類のパケットがやり取りされる。

表 35 楽器機間の同期通信で使用するヘッダの一覧

ヘッダ	送信元 → 送信先	概要
'S'	指揮デバイス → 全楽器機	演奏開始指示, 受信した各機は状態を初期化する
'E'	指揮デバイス → 全楽器機	演奏終了指示, 受信した各機は状態を停止状態に戻す
'V'	指揮デバイス → 全楽器機	音量段階の通知
'J'	スレーブ → マスター	同期への参加予告
'M'	スレーブ → マスター	レーザー検知による BPM 提案
'N'	マスター → 各スレーブ	多数決により決定した確定 BPM のブロードキャスト
'T'	スレーブ → マスター	自身の globalTick の報告 (diff 算出依頼)
'R'	マスター → スレーブ	T 受信時点での diff (Tick 差分) の返信
'O'	マスター → スレーブ	演奏開始タイミング (startGlobalTick) の指示

続いて、レーザー光の検知処理について説明する。各 Arduino は、中継機デバイスの回転周期を CdS セルで検知するために、2.2.3 項で説明した LaserDetector クラスと同一の適応しきい値方式のロジックを実装した SyncManager クラスを備えている。検知したパルス間隔から求めた実測 BPM は、マスターでは自身の BPM 推定値として、スレーブでは'M' 通信による BPM 提案として用いられる。検知ロジックの詳細な実装は 2.2.3 項および付録に示すプログラム全文を参照されたい。

次に、ドラム機（マスター）における同期状態管理について説明する。マスターは、state という状態変数によって管理される 5 段階の状態遷移に沿って動作する。各状態の内容を表 36 に示す。

表 36 ドラム機（マスター）の状態遷移一覧

state	内容
0	BPM 安定待ち。レーザー検知による BPM 推定値が MIN_STABLE_COUNT 回連続で一致するまで待機する
1	楽器と同期中。参加を予告したスレーブ全機について、'T' 受信時に返す diff が閾値以内に収まるまで待機する
2	同期完了・演奏開始前。各スレーブへの'O' 通信の送信タイミングを監視しつつ、ドラム自身の演奏開始タイミングを待つ
3	演奏中。elapsedPlayTick を進めながらドラム自身の楽譜を再生する
4	システム停止中。'S' 通信を受信するまで何も行わない

続いて、ドラム機（マスター）で使用する主な変数の仕様を表 37 に示す。

表 37 ドラム機（マスター）で使用する主な変数の仕様一覧

変数名	型	説明
state	int	現在の同期状態（0～4）を保持
globalTick	uint32_t	マスターを基準とする全体共通の Tick カウンタ
currentSongId	int	現在演奏中の楽譜 ID を保持. 'S' 受信のたびにローテーションする
elapsedPlayTick	int32_t	state2 移行後に進行する演奏開始からの経過 Tick
isDrumPlaying	bool	ドラム自身が演奏中かどうかを保持
isInstJoin[]	bool[4]	各楽器が同期に参加する予告（'J' 受信）をしたかを保持
isInstConnected[]	bool[4]	各楽器が'T' 通信により実際に接続済みであるかを保持
lastReceivedDiff[]	int32_t[4]	各楽器から'T' 受信時に算出した直近の diff を保持
receivedBpmLevels[]	uint8_t[4]	自身および各スレーブから提案された BPM 段階を集約する配列
currentBpmLevel	uint8_t	多数決により確定した現在の BPM 段階
oSentCount[]	uint8_t[10]	各演奏開始イベントに対する'O' 通信の送信済み回数（最大 3 回）
startSequence[][][]	int	曲 ID ごとの各楽器の演奏開始 Tick と楽器 ID を定義する配列

続いて、ドラム機（マスター）で使用する主な関数の仕様を表 38 に示す。

表 38 ドラム機（マスター）で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Wi-Fi 接続と UDP 通信の初期化を行う
loop	void	なし	UDP 受信処理と state に応じたステートマシンの実行を毎回行う
checkSyncReady	bool	なし	参加予告のあった全スレーブが接続済みかつ diff が閾値以内であれば true を返す
processOCommands	void	なし	各演奏開始イベントについて、タイミングが近づいたスレーブへ'O' 通信を送信し、ドラム自身の演奏開始も管理する
sendOPacket_Single	void	uint8_t instId, uint8_t songId, uint32_t start-GlobalTick	指定したスレーブへ'O' 通信（演奏開始タイミング指示）を送信する
sendNPacket	void	uint8_t bpm	確定 BPM を全スレーブへ'N' 通信でブロードキャストする
sendRPacket	void	uint8_t instId, int32_t diff	'T' 受信元のスレーブへ diff を'R' 通信で返信する
updateTick	void	なし	tickIval ごとに globalTick を進め、48Tick ごとの多数決 BPM 決定や自身の楽譜再生を行う
setBpm	void	uint8_t bpm	確定 BPM から tickIval を再計算する
determineSystemBpmLevel	uint8_t	なし	参加中の全楽器の BPM 提案から中央値を求め、システム全体の BPM 段階を決定する
sendPacket	void	なし	curTick に到達した音符を Note 配列か

以下では、マスターの同期制御における中核部分について、実際のコードとともに説明する。まず、'T' 通信（スレーブからの Tick 報告）の受信処理について説明する。この処理は、スレーブの globalTick とマスター自身の globalTick との差である diff を算出し、'R' 通信で即座に返信する。該当部分をコード 45 に示す。

コード 45 drum_0706.ino の loop 関数（'T' 通信受信部分）

```
1 case 'T':
2   if (pktSize >= 6) {
3     uint32_t slaveTick =
4       ((uint32_t)packetBuffer[1] << 24) |
5       ((uint32_t)packetBuffer[2] << 16) |
6       ((uint32_t)packetBuffer[3] << 8) |
7       (uint32_t)packetBuffer[4];
8     uint8_t id = packetBuffer[5];
9
10    if (id < MAX_INSTRUMENTS && id != drumInstId) {
11      isInstConnected[id] = true;
12      if (isInstJoin[id] != true) isInstJoin[id] = true;
13      int32_t diff = (int32_t)(globalTick - slaveTick);
14      lastReceivedDiff[id] = diff;
15
16      sendRPacket(id, diff);
17    }
18  }
19  break;
```

次に、同期完了判定を行う checkSyncReady 関数について説明する。この関数は、参加予告（'J' 受信）のあった全スレーブについて、接続済みであり、かつ直近の diff が tickIval から算出したしきい値以内であるかを確認し、1 台でも条件を満たさなければ false を返す。checkSyncReady 関数をコード 46 に示す。

コード 46 checkSyncReady 関数

```
1 bool checkSyncReady() {
2   bool expectedToJoin = false;
3
4   for (uint8_t i = 1; i < MAX_INSTRUMENTS; i++) {
5     if (isInstJoin[i]) {
6       expectedToJoin = true; // 参加予定の楽器が少なくとも1台いる
7       float SYNC_READY_THRESHOLD = 50 / tickIval;
8
9       // 参加予定だが未接続、または同期のズレが大きい場合は「まだ準備未完了」
10      if (!isInstConnected[i] || abs(lastReceivedDiff[i]) > SYNC_READY_THRESHOLD) {
11        return false;
12      }
13    }
14  }
15  return expectedToJoin;
```

16 }

最後に、各楽器の演奏開始タイミングを管理する processOCommands 関数について説明する。この関数は、startSequence に定義された各演奏開始イベントについて、該当楽器がドラム自身であれば elapsedPlayTick が開始 Tick に達した時点で演奏を開始し、スレーブであれば開始 Tick の O_ADVANCE_TICKS (96Tick) 前から 3 段階に分けて'O' 通信を送信することで、パケットロスに備えた冗長性を確保している。該当部分をコード 47 に示す。

コード 47 processOCommands 関数 (要部)

```
1  if (instId == drumInstId) {
2      if (elapsedPlayTick >= evTick) {
3          if (!isDrumPlaying) {
4              isDrumPlaying = true;
5              curTick = songStartParams[currentSongId].startTick;
6              scoreIdx = songStartParams[currentSongId].startNoteIdx;
7              state = 3;
8          }
9          oSentCount[i] = 3;
10     }
11 } else {
12     if (isInstJoin[instId] && isInstConnected[instId]) {
13         uint32_t startGlobalTick = (uint32_t)((int32_t)globalTick + ticksUntilStart);
14         if (ticksUntilStart <= O_ADVANCE_TICKS && oSentCount[i] == 0) {
15             sendOPacket_Single(instId, currentSongId, startGlobalTick);
16             oSentCount[i] = 1;
17         } else if (ticksUntilStart <= O_ADVANCE_TICKS * 2 / 3 && oSentCount[i] == 1) {
18             sendOPacket_Single(instId, currentSongId, startGlobalTick);
19             oSentCount[i] = 2;
20         } else if (ticksUntilStart <= O_ADVANCE_TICKS / 3 && oSentCount[i] == 2) {
21             sendOPacket_Single(instId, currentSongId, startGlobalTick);
22             oSentCount[i] = 3;
23         }
24     }
25 }
```

次に、楽器機 (スレーブ) における同期処理について説明する。スレーブは、state が 0 (BPM 安定待ち)・1 (マスターと同期中)・2 (演奏中)・4 (停止中) の 4 状態で動作し、'R' 通信で受け取った diff をもとに自身の tickIval を補正することでマスターとの同期を保つ。続いて、楽器機 (スレーブ) で使用する主な変数の仕様を表 39 に示す。

表 39 楽器機（スレーブ）で使用する主な変数の仕様一覧

変数名	型	説明
baseTickIval	uint32_t	確定 BPM から算出した基本の 1Tick あたりの時間 [μ s]
tickIval	uint32_t	同期補正を反映した実際に使用する 1Tick あたりの時間 [μ s]
ticksToCorrect	int32_t	補正処理が残っている Tick 数
lastDiff	int32_t	直近に受信した diff. BPM 変化時の補正再計算に使用
drumNotStarted	bool	マスターがまだ globalTick の加算を開始していないかを保持
waitingForStart	bool	'O' 通信を受信し、演奏開始タイミングを待機中かどうかを保持
startGlobalTick	uint32_t	マスターから指示された演奏開始 globalTick

続いて、楽器機（スレーブ）で使用する主な関数の仕様を表 40 に示す。

表 40 楽器機（スレーブ）で使用する主な関数の仕様一覧

関数名	戻り値	引数	説明
setup	void	なし	Wi-Fi 接続と UDP 通信の初期化を行う
loop	void	なし	UDP 受信処理 ('R'・'O'・'N'・'S'・'E'・'V') と state に応じたステートマシンの実行を毎回行う
applyDiffCorrection	void	int32_t diff	受信した diff の大きさに応じて tickIval を加減速し、マスターとの同期を図る
setBpm	void	uint8_t bpm	確定 BPM から baseTickIval を再計算する
sendTPacket	void	なし	自身の globalTick を 'T' 通信でマスターへ送信し、diff の算出を依頼する
sendJPacket	void	なし	同期への参加予告を 'J' 通信でマスターへ送信する
sendMPacket	void	uint8_t bpm	レーザー検知による BPM 提案を 'M' 通信でマスターへ送信する
updateTick	void	なし	tickIval ごとに globalTick を進め、補正カウントダウンや自身の楽譜再生を行う
sendPacket	void	なし	curTick に到達した音符を Note 配列から取得し、自身の Processing ヘシリアル送信する

以下では、スレーブの同期補正における中核部分について、実際のコードとともに説明する。まず、'R' 通信（マスターからの diff 返信）の受信処理について説明する。この処理は、パケットの往復時間から通信遅延を見積もって diff を補正したうえで、マスターがまだ動作していない場合の特別処理を経て、通常時は applyDiffCorrection 関数を呼び出す。該当部分をコード 48 に示す。

コード 48 flute_0706.ino の loop 関数 ('R' 通信受信部分・要部)

```

1 case 'R':
2   if (state == 0 || state == 4) break;
3   if (pktSize >= 5) {

```

```

4      int32_t rawDiff =
5          ((int32_t)(int8_t)packetBuffer[1] << 24) |
6          ((int32_t)packetBuffer[2] << 16) |
7          ((int32_t)packetBuffer[3] << 8) |
8          (int32_t)packetBuffer[4];
9
10     uint32_t rtt = micros() - lastTPacketMicroTime;
11     uint32_t latencyMicros = rtt / 2; // 片道遅延
12     int32_t latencyTicks = (baseTickIval > 0)
13         ? (int32_t)((latencyMicros + (baseTickIval / 2)) / baseTickIval) : 0;
14     int32_t diff = rawDiff - latencyTicks; // 通信遅延を差し引いた真のズレ
15
16     lastDiff = diff;
17     if (state >= 1) {
18         if (ticksToCorrect == 0 || abs(diff) >= 24) {
19             applyDiffCorrection(diff);
20         }
21     }
22 }
23 break;

```

次に、applyDiffCorrection 関数について説明する。この関数は、diff の大きさに応じて補正の強さを 4 段階に切り替える。ズレが小さい場合は不感帯として何もせず、大幅なズレの場合は tickIval を 2 倍または 2/3 倍にして急速に補正し、中程度のズレの場合は次の 48Tick で吸収するように比率補正を行う。applyDiffCorrection 関数をコード 49 に示す。

コード 49 applyDiffCorrection 関数

```

1 void applyDiffCorrection(int32_t diff) {
2     if (baseTickIval == 0) return;
3     float SYNC_READY_THRESHOLD = 50 / baseTickIval;
4
5     if (abs(diff) <= SYNC_READY_THRESHOLD) {
6         // 不感帯：ズレが小さければ何もしない
7         tickIval = baseTickIval;
8         ticksToCorrect = 0;
9     } else if (diff <= -24) {
10        // 大幅に早い → ×2 で -diff Tick間遅らせる
11        tickIval = baseTickIval * 2;
12        ticksToCorrect = -diff;
13    } else if (diff >= 24) {
14        // 大幅に遅い → ×2/3 で ×diff3 Tick間速める
15        tickIval = baseTickIval * 2 / 3;
16        ticksToCorrect = diff * 3;
17    } else {
18        // 小さいズレ → 次の48Tickでdiffを吸収する比率補正
19        tickIval = (uint32_t)((48UL * baseTickIval) / (48 + diff));
20        ticksToCorrect = 48 + diff;
21    }

```

最後に、'O' 通信（マスターからの演奏開始指示）の受信処理について説明する。この処理は、指示された startGlobalTick と自身の globalTick を比較し、まだ到達していなければ waitingForStart を立てて待機し、すでに超過していれば超過分だけ curTick をオフセットして即座に演奏へ合流する。なお、'O' 通信の受信および状態遷移に関する処理の詳細な実装は付録に示すプログラム全文を参照されたい。

プログラム全文は付録に示す。

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは、6 名で実施した。

3.2 報告者の担当範囲と技術的課題

本節では、担当したハードウェアの詳細設計と LEDMatrix での BPM 表示を行うプログラムについて説明する。

3.3 部品一覧

次に、楽器デバイスが受光するための部品について表 41 に示す。光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、観覧車デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するためそれぞれに Arduino を使用し、光センサおよびブレッドボードも 4 台用意する。

表 41 楽器デバイスに必要な機器

機材	品番	個数 [個]
Arduino UNO R4	-	4
CdS セル	GL5516	4
ブレッドボード	-	4
ジャンパーワイヤー	-	適宜

3.4 指揮デバイス

3.5 観覧車型中継機デバイス

本デバイスのモータ回転制御および LED マトリクス表示を行う制御プログラムの全文は付録 (FerrisWheel.ino, Display.h, Display.cpp, MotorControl.h, MotorControl.cpp) に示す。

3.6 レーザー光送信装置

レーザー光送信装置の詳細設計について説明する。図 19 について、回路図の通りにボタン電池とレーザーをはんだ付けする。これらを観覧車のリングに 45 度間隔に装着する。電源としては図 19 に示すようにボタン電池からの 3V 給電を行う。ここで、レーザーについて、データシートより定電流源回路を組み込んであるため、現在の設計であれば外部抵抗は必要とせず、素子を安全に利用することができる [12]。

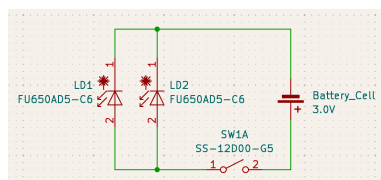


図 19 レーザー回路図

3.7 楽器デバイス受光部装置

楽器デバイスの受光部の回路図を図 20 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を Arduino で処理することで光を検知する仕様である。

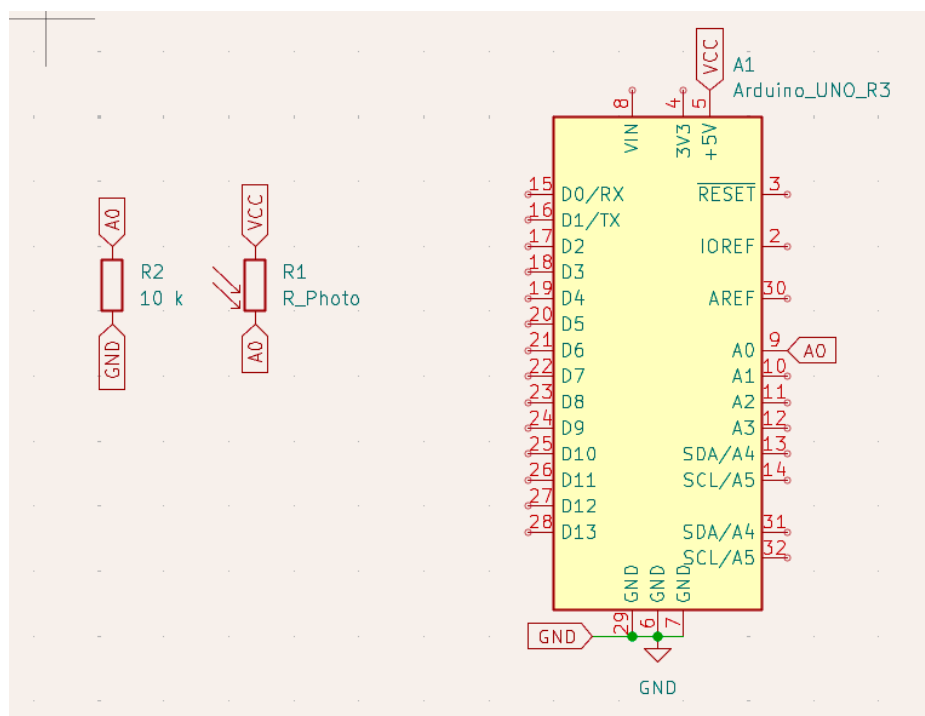


図 20 楽器デバイスの受光部の回路図

4 おわりに

これは $\text{T}_{\text{E}}\text{X}$ による報告書作成のテンプレートでした.

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場(～2031年)、市場規模(パフォーマンス、同期、デジタル収益)・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] 谷口由貴, “ストーリーミングサービスが生み出した新たな音楽消費サイクル,” 博報堂 The Hakuhodo, <https://www.hakuhodo.co.jp/magazine/61505/>, 2019/9/12, 2026/7/7 参照.
- [5] Martin Kreuzer, Melanie Wald-Fuhrmann, Christian Weining, Martin Tröndle, and Hauke Egermann, “Western Classical Music Concerts Are More Immersive, Intellectually Stimulating, and Social, When Experienced Live Rather Than in a Digital Stream: An Ecologically Valid Concert Study on Different Modes of Liveness,” *Music & Science*, vol.8, 2025.
- [6] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” *PLOS ONE*, vol.13, no.2, p.e0191846, 2018.
- [7] Martin, R., and Nielsen, N. Enacting Musical Aesthetics: The Embodied Experience of Live Music. *Music & Science*, 7, 2024
- [8] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [9] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.7, 2026, p.382-385, 2021
- [10] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.
- [11] Control.com, “DC Motor Speed Control,” *Lessons In Electric Circuits*, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, (参照 2026-05-04).
- [12] 秋月電子通商, “fu650ad5-c6 データシート,” <https://akizukidenshi.com/goodsaffix/fu650ad5-c6.pdf>, (参照 2026-06-30).